115-1 TAICA《生成式 AI：文字與圖像生成的原理與實務》

02 呆萌型 AI 機器人

神經網路與函數學習機

政治大學應用數學系 蔡炎龍

01.

打造呆萌型 AI 機器人

現代的 AI 模型只是一個「呆萌型 AI 機器人」

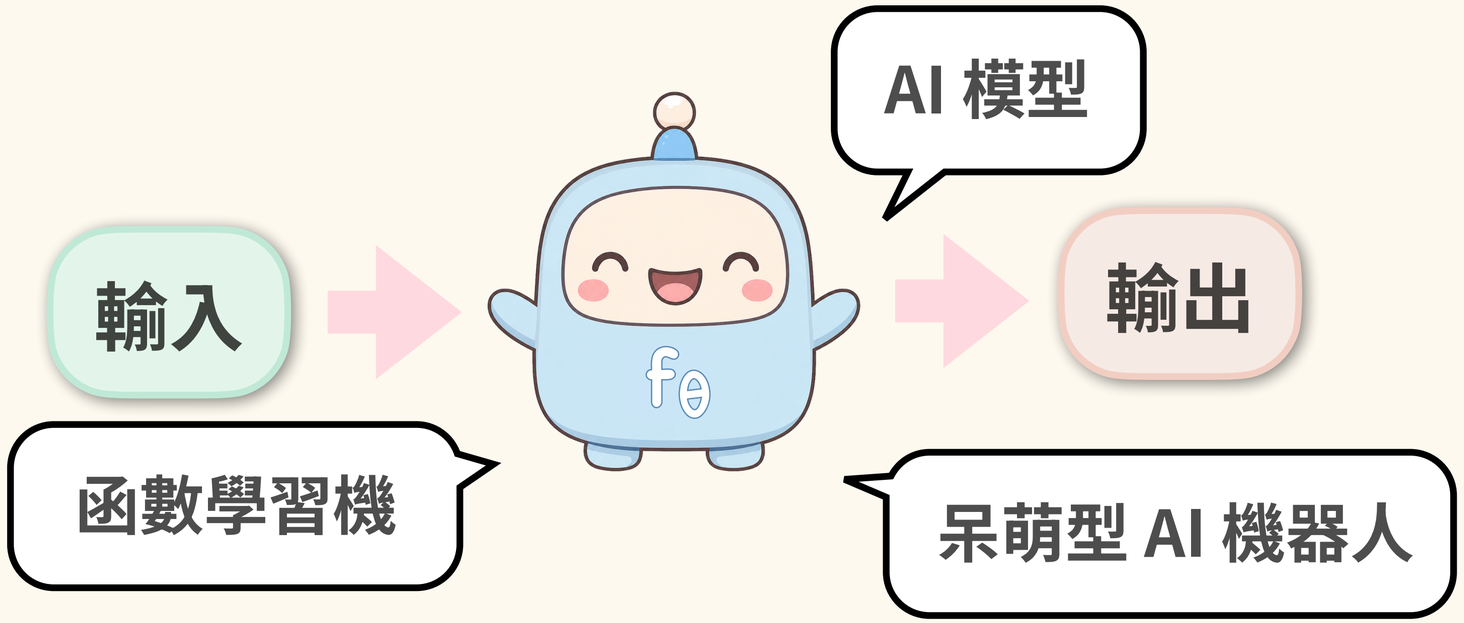
輸入

輸出

不會自己知道要做什麼，需要我們告訴它：輸入是什麼、輸出是什麼。

不會分析、沒有意識、不知道自己在做什麼。

呆萌型 AI 機器人的種種名稱



就是知道**輸入**是什麼、**輸出**是長什麼樣子

注意輸入和輸出都需要是數字

輸入和輸出都需要
數據 (一堆數字)。

我想預測某支股價會漲還是會跌

這時輸入是什麼、輸出是什麼呢？

這個 AI 模型的輸入是什麼、輸出是什麼

比如說過去 20 天的數據

y_1 漲

y_2 跌

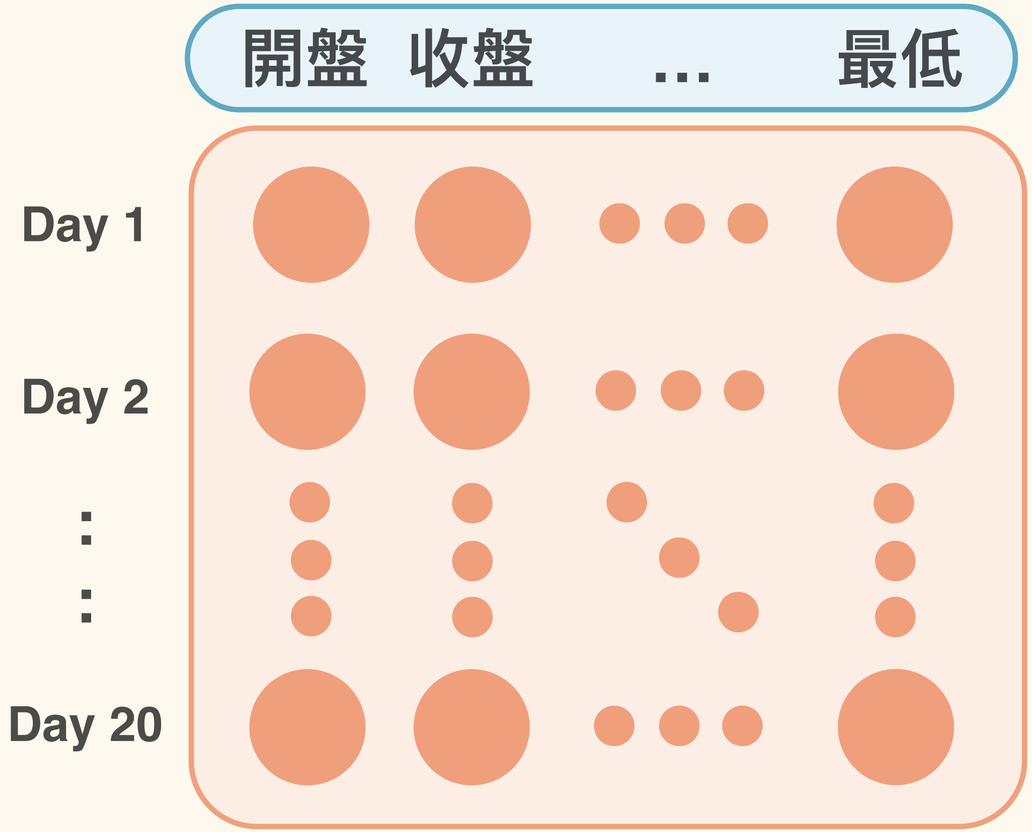
輸入

輸出

想一想每天要什麼數據呢？

即使是同樣目標的模型，每個人想法不一樣

某股過去 20 天的數據



總記 $20 \times N$ 個特徵

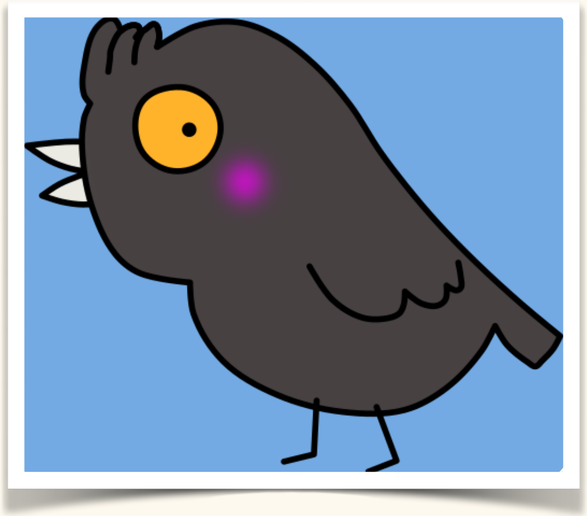
更精確的說, 輸入或輸出是一個張量

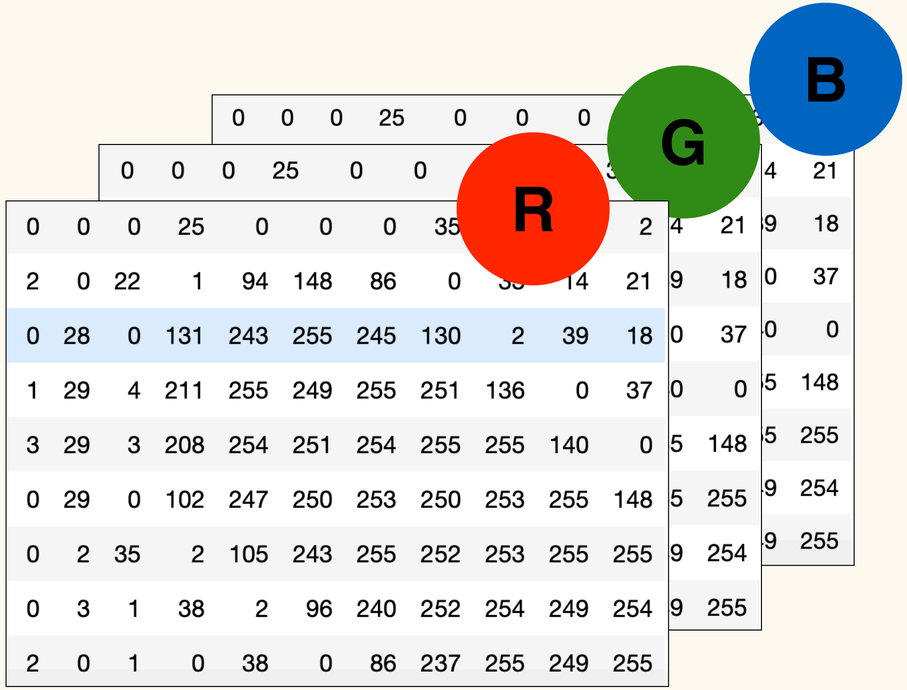
純量 $x = 9487$ — 0 階 tensor

向量 $x = [9,4,8,7]$ — 1 階 tensor

矩陣 $x = \begin{bmatrix} 9 & 4 \\ 8 & 7 \end{bmatrix}$ — 2 階 tensor

有時是... 3D 矩陣?

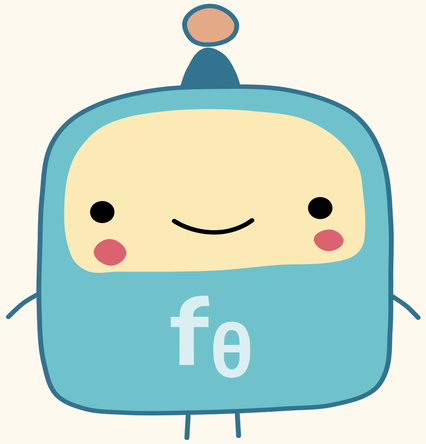




然後我們就沒詞了...

以台灣常見八哥辨識為例

辨識台灣常見三種八哥, 輸出就是三個數字, 分別代表各八哥的「分數」。



y_1 土八哥

y_2 白尾八哥

y_3 家八哥

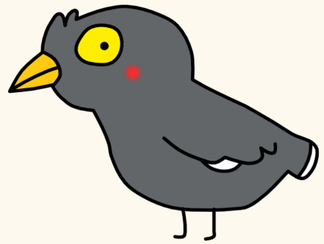
輸出怎麼看成數字呢?

1

土八哥

$$\begin{bmatrix}1\\0\\0\end{bmatrix}$$

2



白尾八哥

$$\begin{bmatrix}0\\1\\0\end{bmatrix}$$

3

家八哥

$$\begin{bmatrix}0\\0\\1\end{bmatrix}$$

我們常把它做成 one-hot encoding

$$\begin{bmatrix}1\\0\\0\end{bmatrix}$$

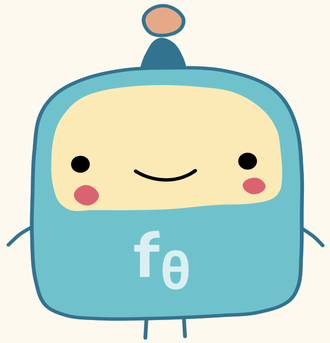
土八哥

白尾八哥

家八哥

然後要準備好很多有正確答案的數據

$\mathbf{x}_1$

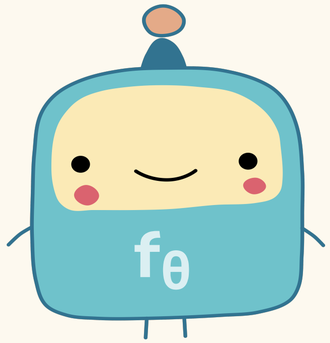


$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

土八哥

$\mathbf{y}_1$

$\mathbf{x}_2$

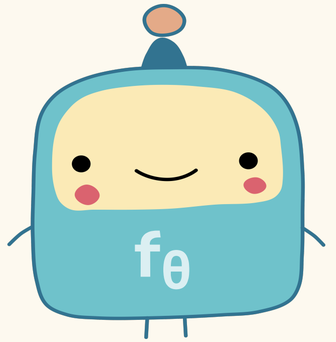


$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$\mathbf{y}_2$

白尾八哥

$\mathbf{x}_3$



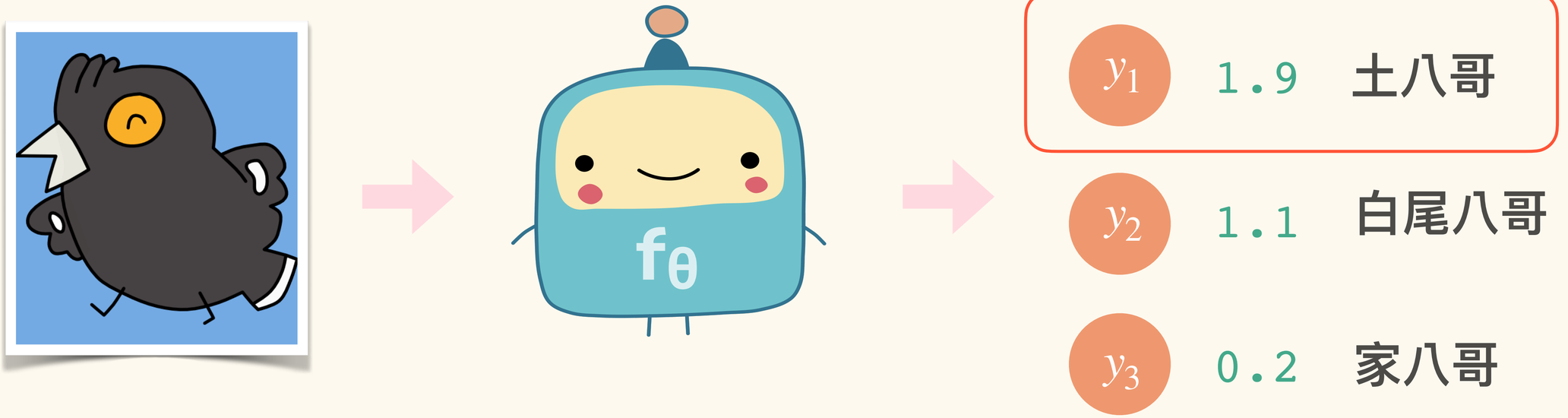
家八哥

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$\mathbf{y}_3$

訓練好了之後...

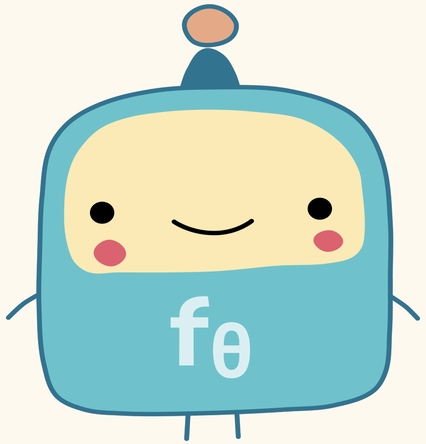
辨識台灣常見三種八哥, 輸出就是三個數字, 分別代表各八哥的「分數」。



這個例子中, 電腦判斷最有可能是土八哥。

一般我們會用 softmax

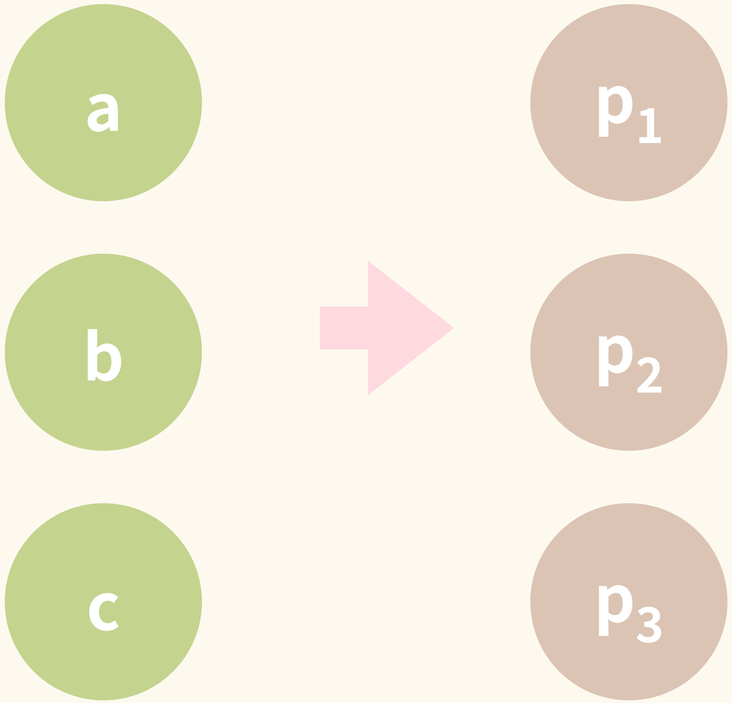
為了看來更有學問, 可以按比例分配, 讓大的還是大、小的還是小, 而加起來等於 1。因為有時輸出可能是小於 0 的負數，會用叫 **softmax** 的轉換, 先將三個數字都轉成大於 0, 再依比例分配。



y_1	1.9	0.61
y_2	1.1	0.28
y_3	0.2	0.11

於是我們知道, 我們的神經網路判斷 61% 的可能是土八哥, 28% 的可能是白尾八哥, 只有 11% 的可能是家八哥。

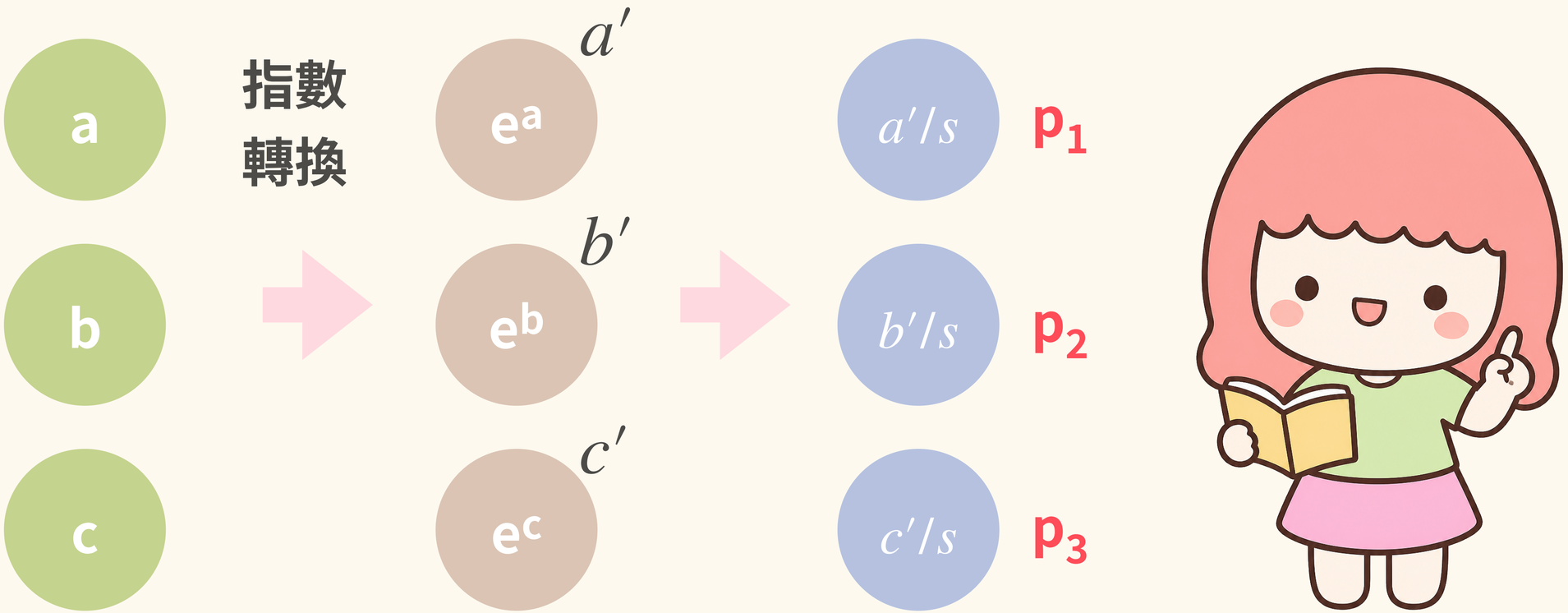
Softmax 概念很簡單



目標是把任意的 a, b, c 三個數字, 轉化為 p_1, p_2, p_3, 保持大小關係, 而且

$$p_1 + p_2 + p_3 = 1$$

Softmax 概念很簡單



$$s = a' + b' + c'$$

如果全部都是正的就容易

先令

$$S = a + b + c$$

然後

$$\alpha = \frac{a}{S}, \beta = \frac{b}{S}, \gamma = \frac{c}{S}$$

Softmax

很自然我們會這麼做:

$$S = e^a + e^b + e^c$$

$$a \quad e^a \quad e^a/S$$

$$b \rightarrow e^b \rightarrow e^b/S$$

$$c \quad e^c \quad e^c/S$$

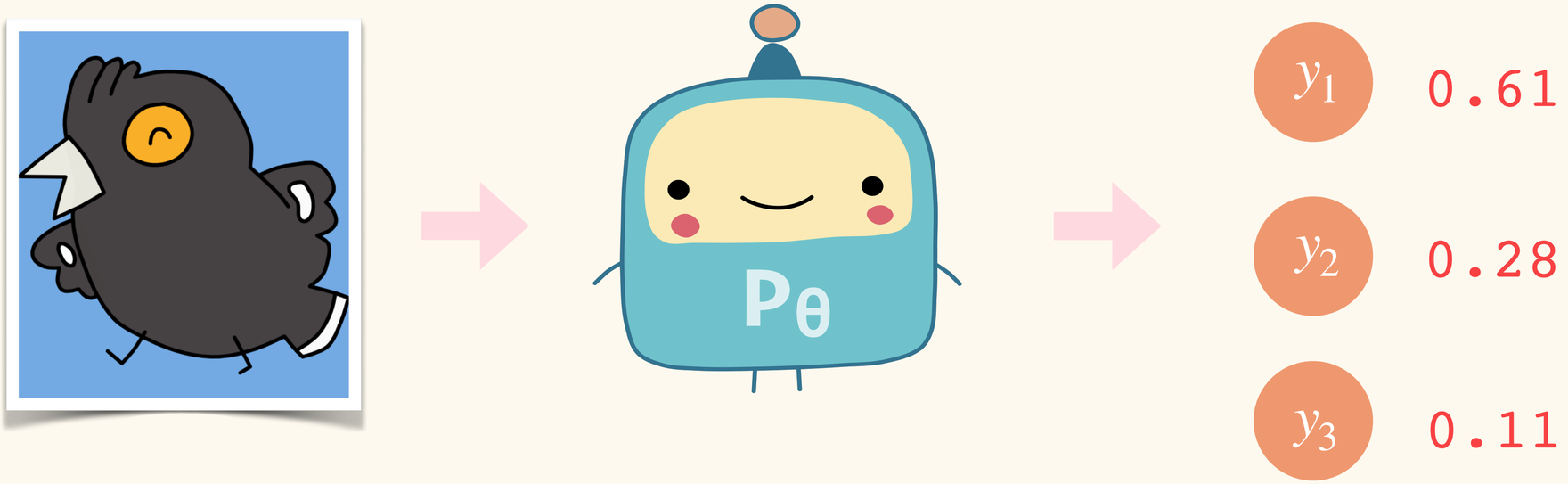
每個數字經指數轉換

按比例

這就叫 **softmax**

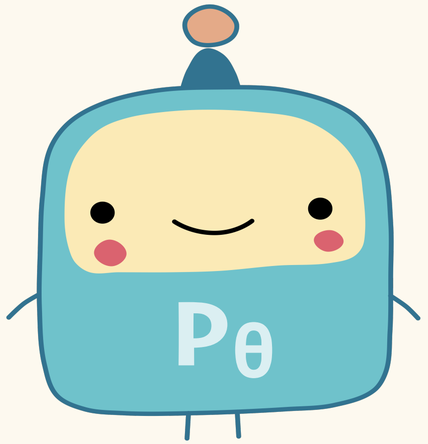
所以會聽人家說我們學的是機率分佈...

因為最後的結果加起來等於 1, 常有人說我們學的是一個機率分佈。



(在某種意涵來說, 這嚇人的成分居多...)

你不要以為剛剛是最嚇人的說法

$\mathbf{x}$ → 

→ $\mathbf{y} = \begin{cases} y_1, & \text{土八哥} \\ y_2, & \text{家八哥} \\ y_3, & \text{白尾八哥} \end{cases}$

一張八哥照片

可以寫成條件機率

$$\mathbf{x} = x$$

$$P_\theta(\mathbf{y} = y_i \mid \mathbf{x} = x) = p_i$$

or

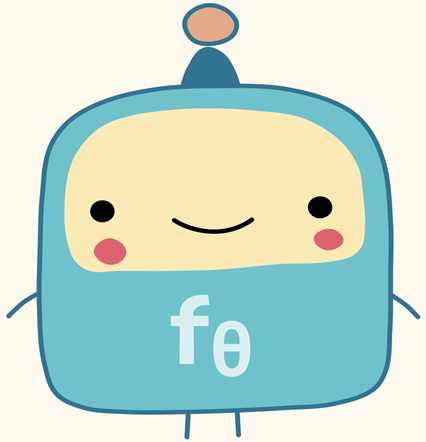
$$P_\theta(y_i \mid x) = p_i$$

深度學習最重要的是要「問個好問題」

深度學習會專注在
「我們怎麼問這個問題?」

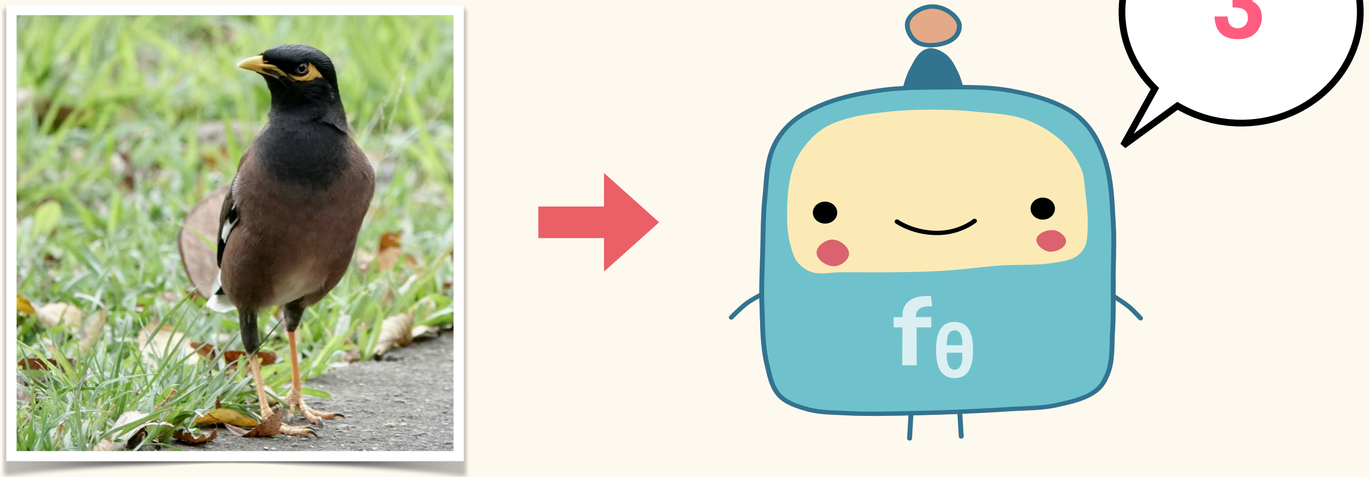
整個判斷由電腦做, 所以
一般要大量資料!

深度學習讓電腦「自己看著辦」，要大量訓練資料



1 個類別我大概需要看 1,000 張照片！

電腦不知自己在做什麼，只依經驗來輸出



注意電腦可不知道這是「家八哥」，它只知道這是第 3 類數據。

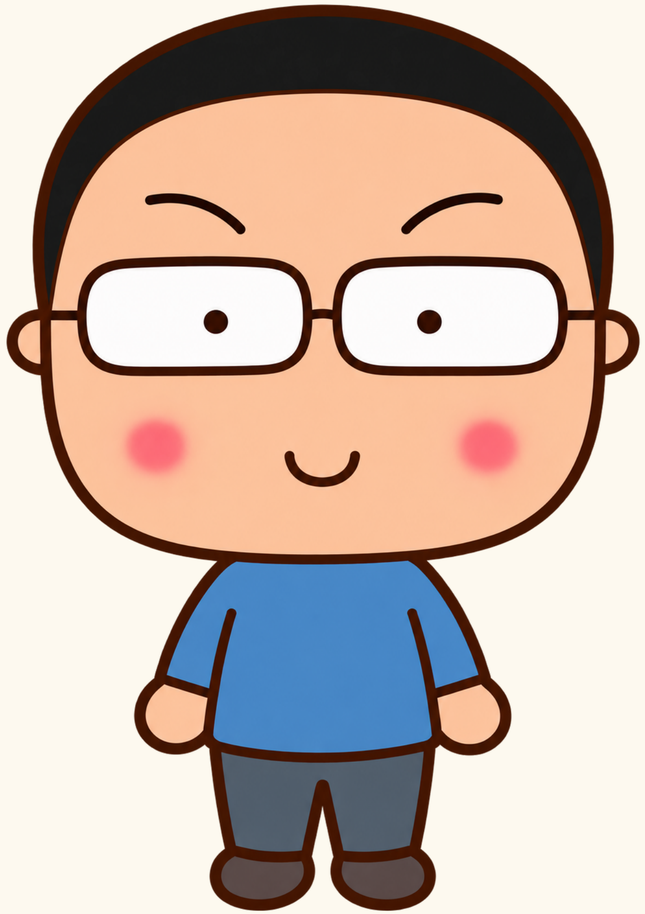
呆一布風五個掌握 AI 的 Dive Points

1. 為什麼我們說，AI 只是一個「呆萌型 AI 機器人」？
2. 你想要呆一布，學哪一個函數?
3. 為什麼我們要說，LLM 生成的都是「幻覺」？
4. 解釋「AI Agents 就是教 AI 怎麼用我們的方式做事」。
5. AI 落地為什麼領域專家才是關鍵？

02.

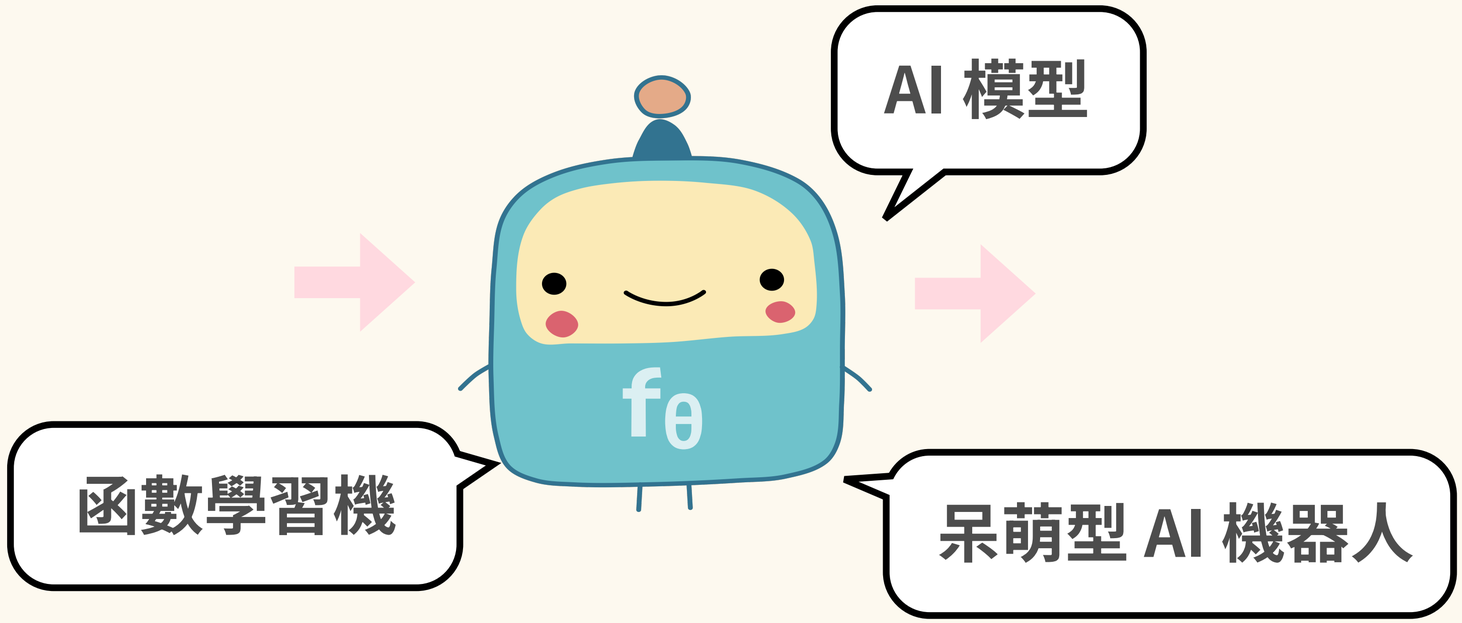
神經網路

深度學習和神經網路



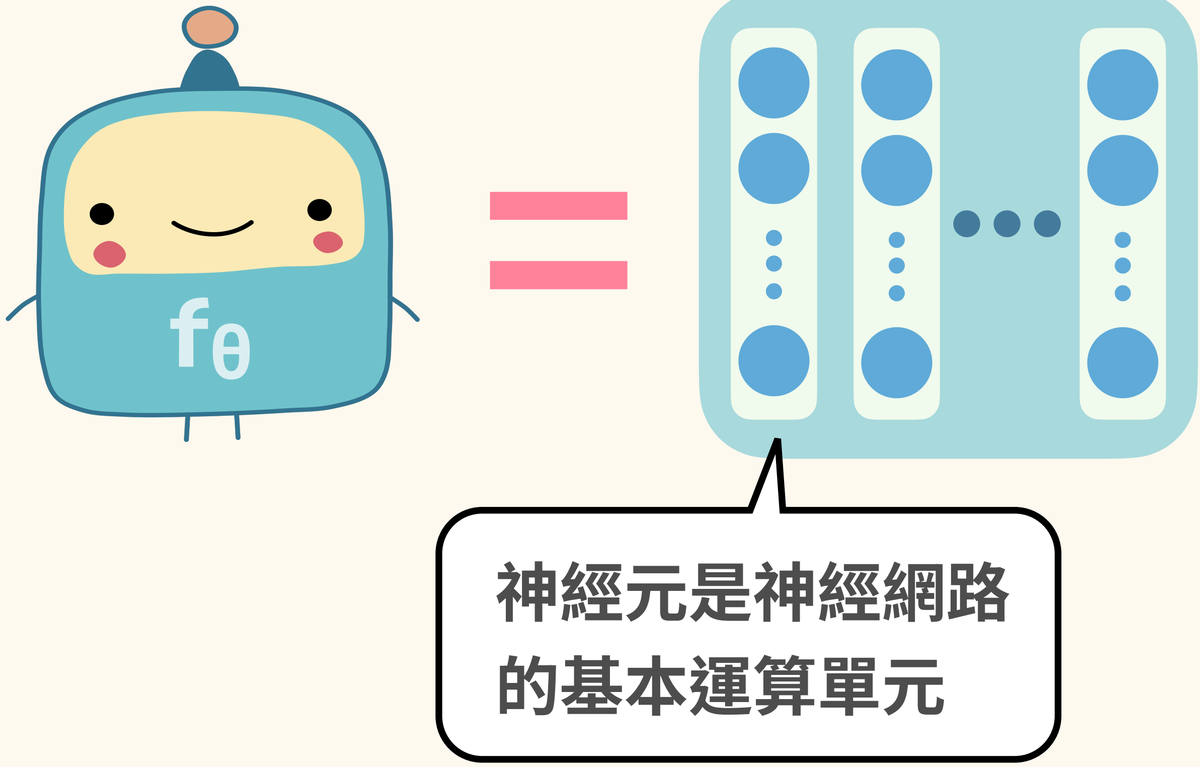
現在 AI 大紅的原因是**深度學習**有很多突破性的發展, 而深度學習的核心是**神經網路**。

記得 AI 模型只是一個呆萌型 AI 機器人



就是知道輸入是什麼、輸出是長什麼樣子

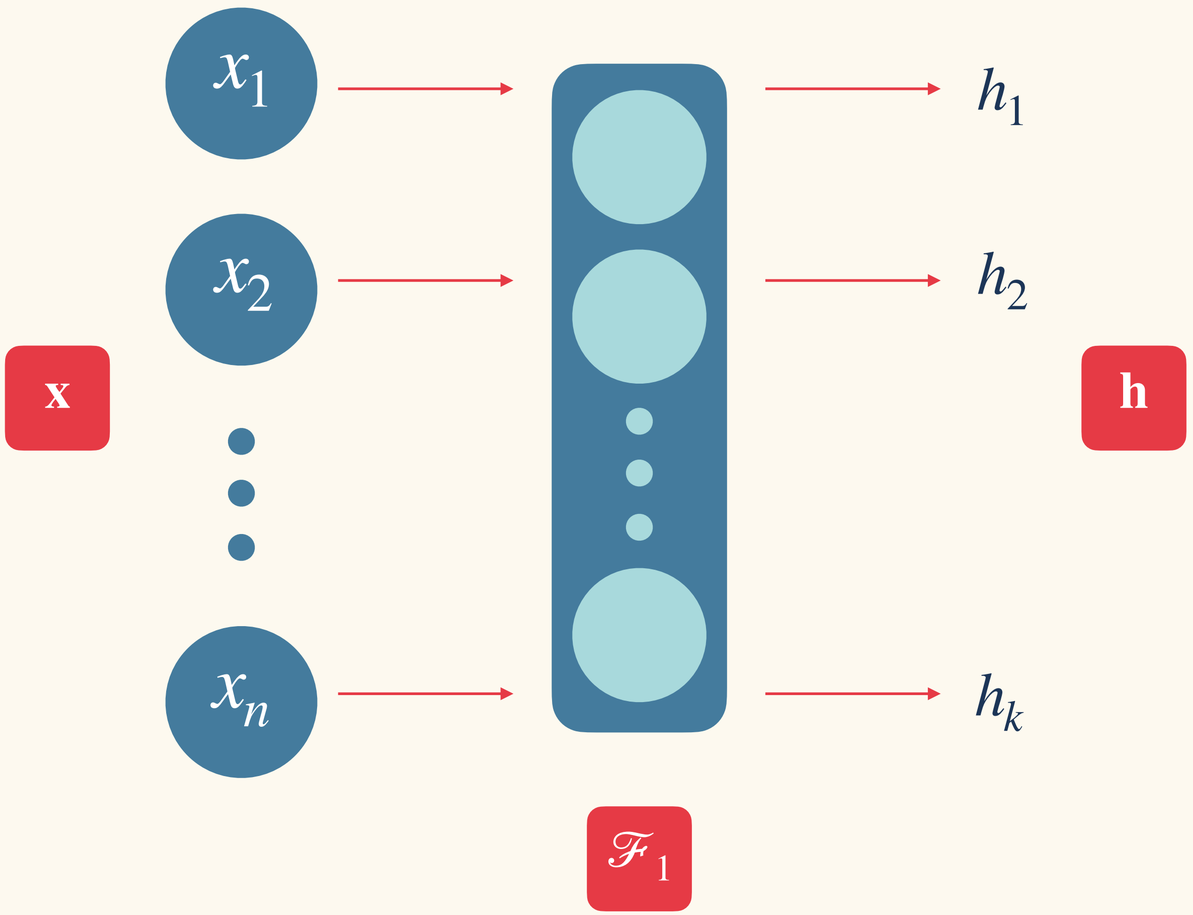
用「神經網路」打造的呆萌型 AI 機器人



基本上就是建一個個隱藏層, 每層放上若干個「神經元」, 放法差不多就 3+1 種:

- DNN: 標準全連結
- CNN: 圖形辨識很強
- RNN: 有記憶的結構
- Transformers

每一種都是由神經元組成



隱藏層基本上就三種選擇:

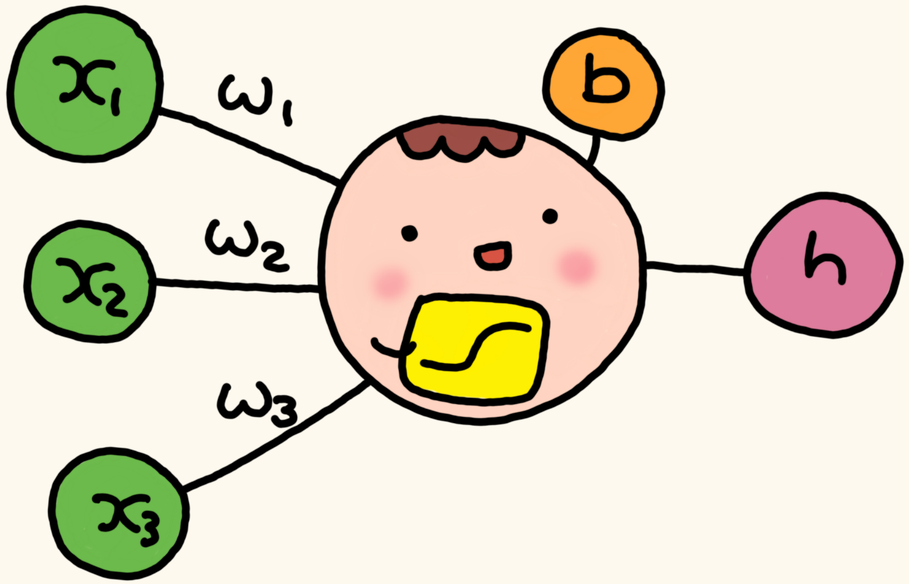
- **全連結層 (Dense)** — DNN
- **卷積層 (Conv)** — CNN
- **遞歸層 (LSTM, GRU)** — RNN

基本上只是**神經元**的連結方式不同, 而神經元是神經網路的**基本運算單元**!

深度學習基本運算單元是「神經元」

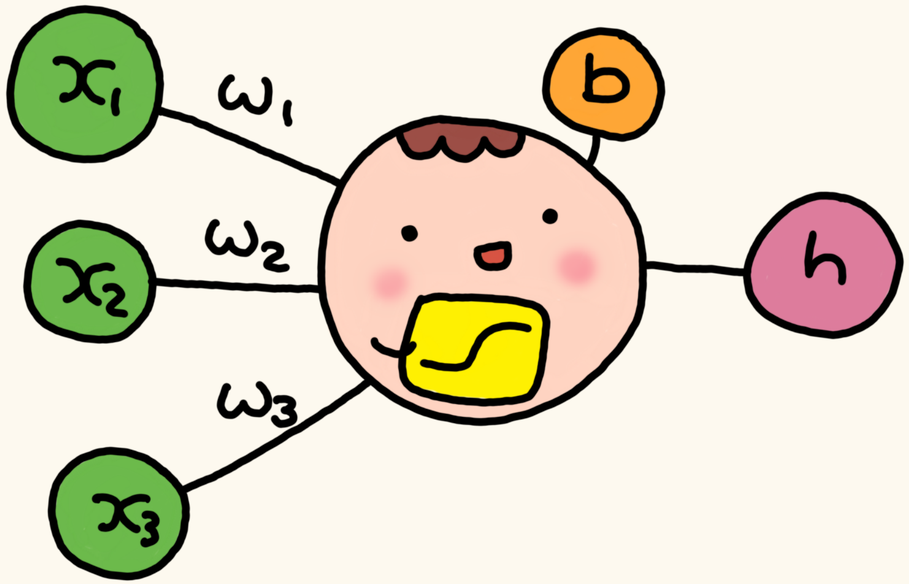
不管哪種神經網路, 神經元就是基本的運算單元, 而每個**神經元的運算方式是一樣的**!

神經元怎麼運作



每個神經元就是接受
若干個刺激輸入, 然後
送出**一個刺激輸出**。

神經元怎麼運作

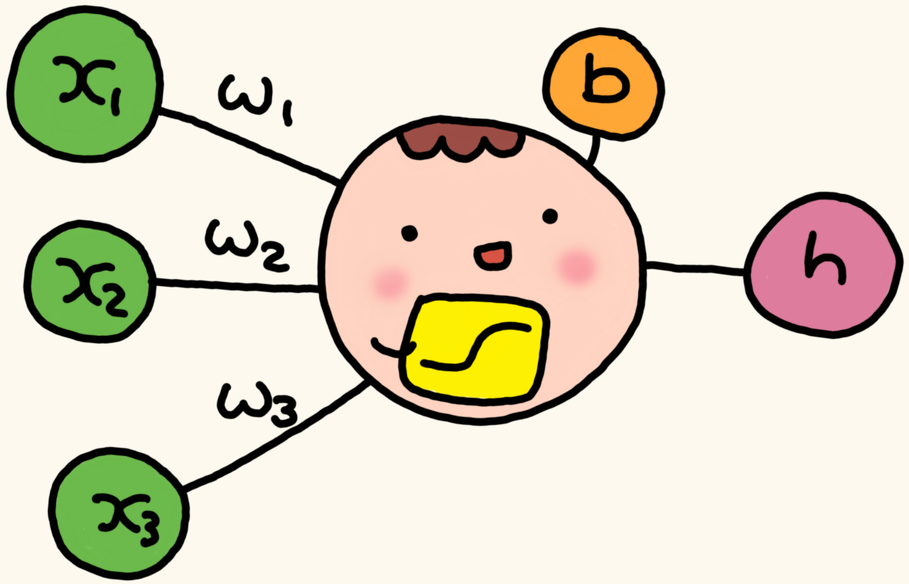


我們先要這個神經元接受到的**總刺激**。

$$w_1x_1+w_2x_2+w_3x_3$$

w_1, w_2, w_3 稱為**權重**, 是學來的!

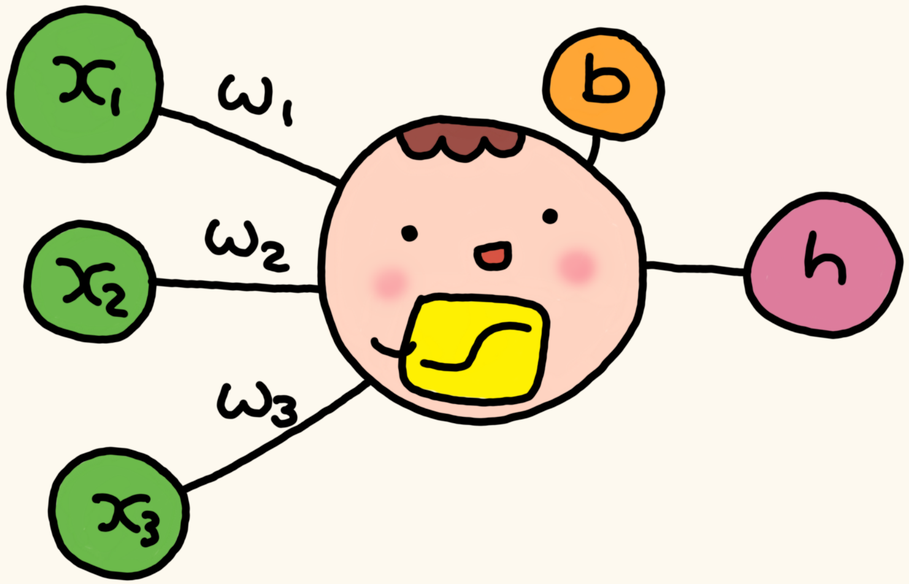
神經元怎麼運作



接著加上**偏值 (bias)**, 做一個基準的調整。不管是權重、偏值都是要**經過學習得到的**! 這裡我們估且稱調整後的總刺激。

$$\sum_{i=1}^{3} w_i x_i + b$$

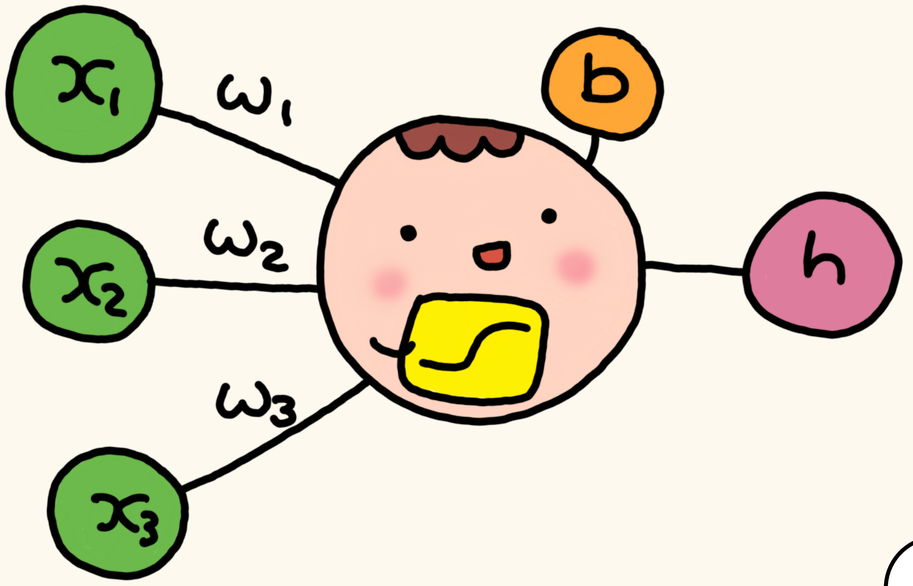
神經元怎麼運作



目前的計算都是線性的, 即使經所有神經元算還是線性的, 因此我們需經一個激發函數 (activation function) 轉換再送出。

$$\varphi\left(\sum_{i=1}^{3} w_i x_i + b\right) = h$$

神經元怎麼運作



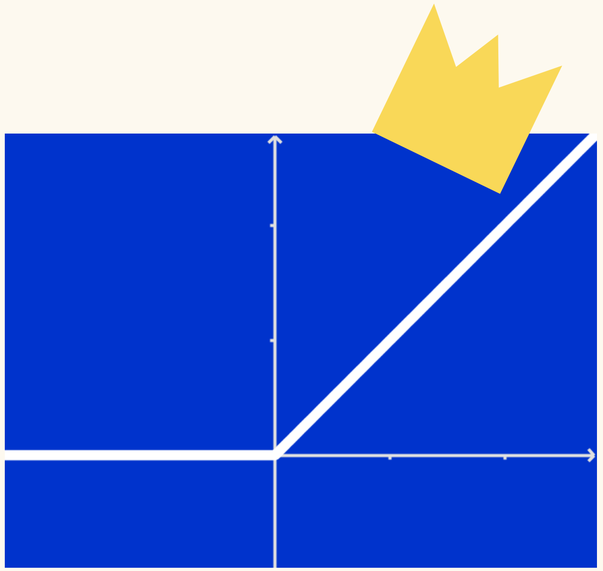
$$\varphi\left(\sum_{i=1}^{3} w_i x_i + b\right) = h$$

$$\theta = \{w_1, w_2, w_3, b\}$$

這是要學習的參數

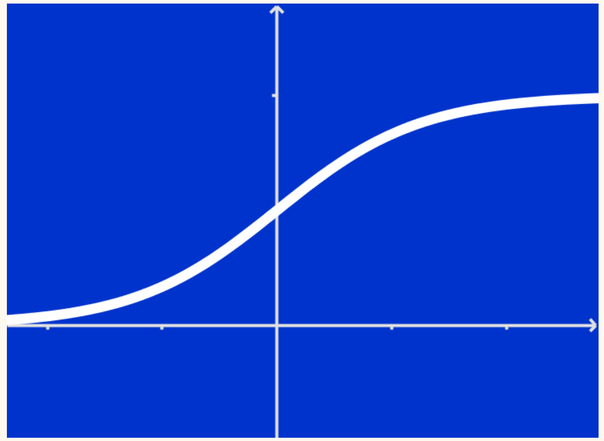
激發函數

以下我們介紹幾個有名的激發函數。



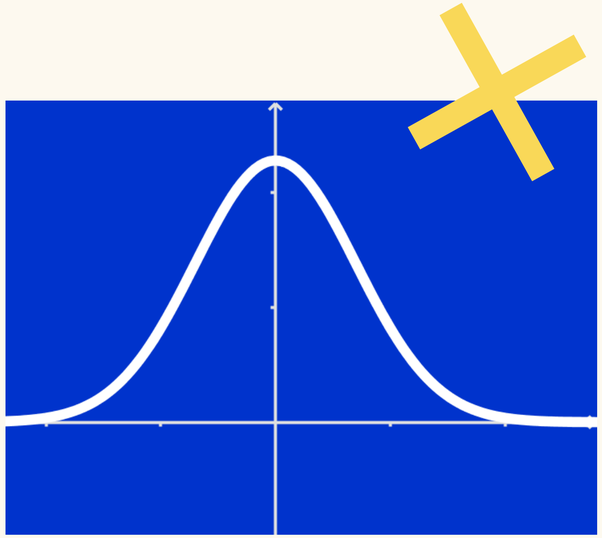
ReLU

本世紀新寵!



Sigmoid

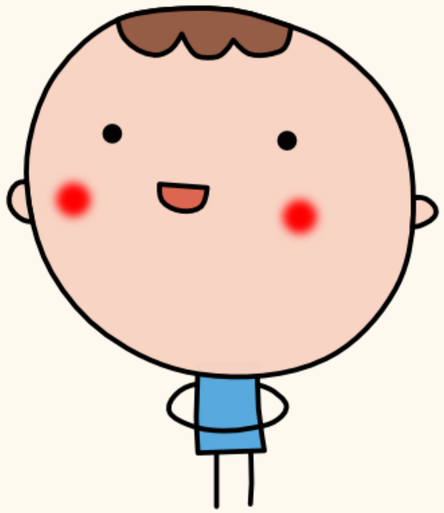
感覺應該很接近人類神經元的動作。



Gaussian

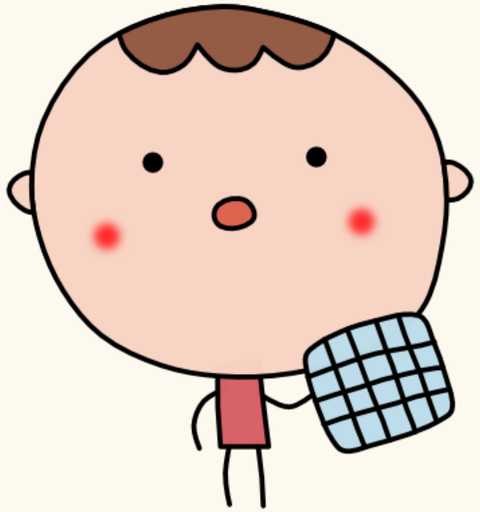
舊時代的懷念, 已很少使用。

深度學習（神經網路）的三大天王



DNN

全面考慮的基本型



CNN

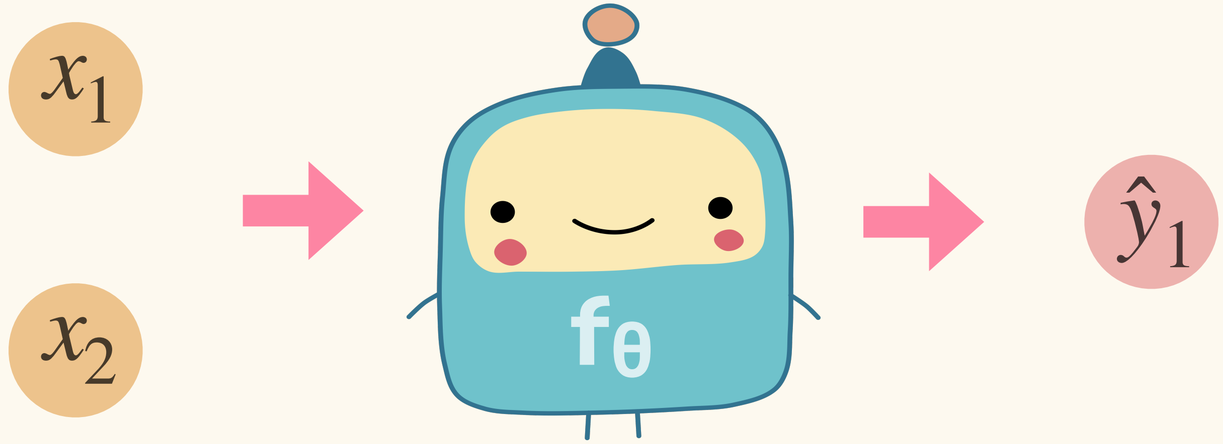
圖形辨識天王

RNN

有記憶的神經網路

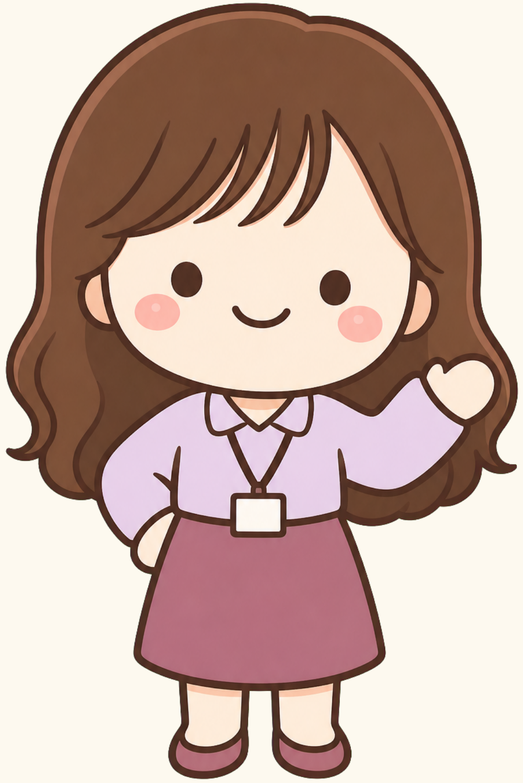
神經網路其實很簡單，
主要只有三種模式。

當呆萌 AI 機器人已經決定好了!



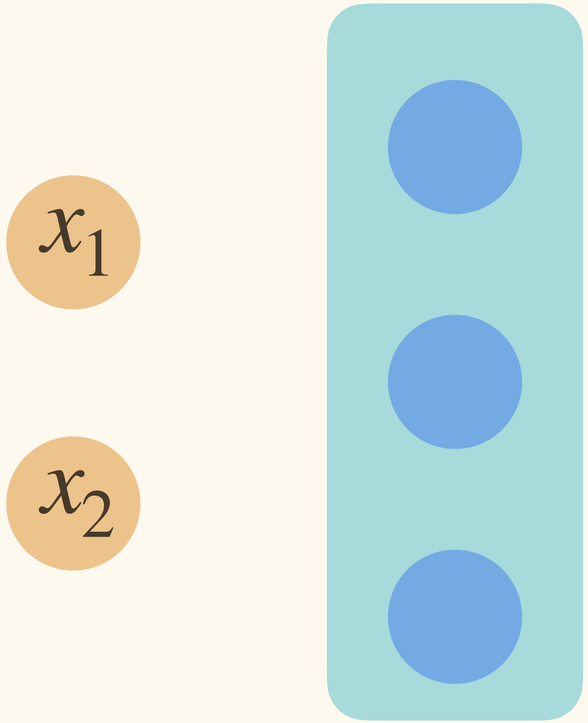
很簡單「兩個輸入、一個輸出」的 AI 模型

以全連結神經網路 (DNN) 為例



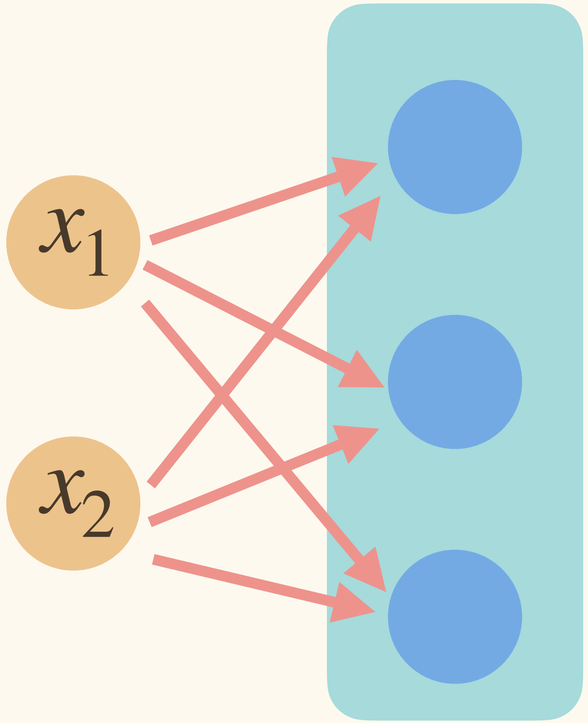
接下來以**全連結神經網路**為例, 示範怎麼打造一個函數學習機!

開始打造神經網路!



DNN 一層只需要
決定有幾個神經元
(比方說 3 個)。

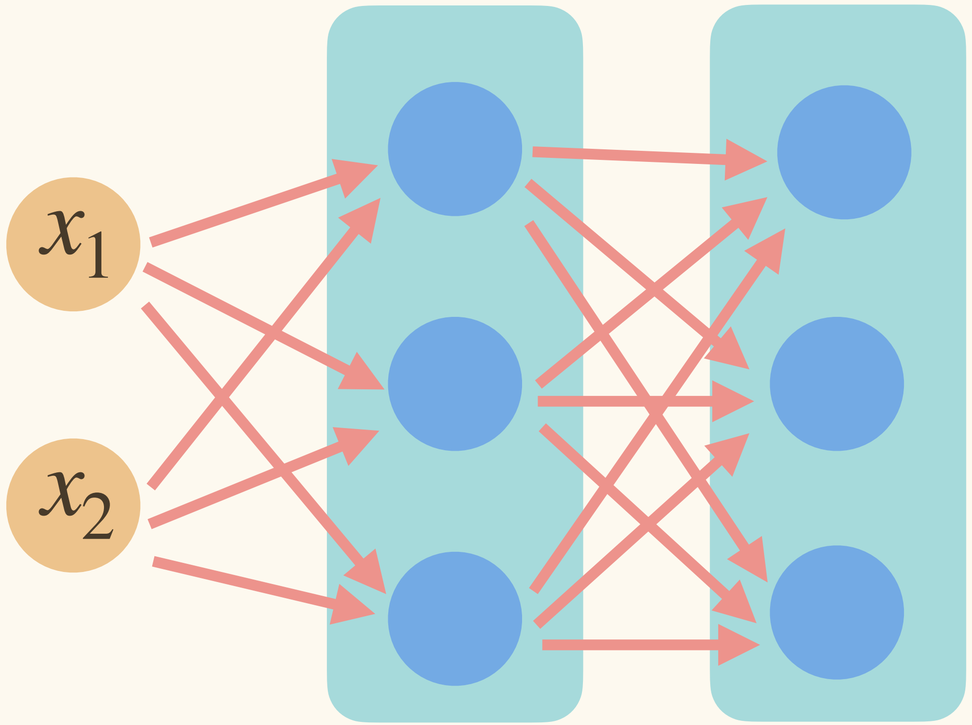
開始打造神經網路!



完全連結, 是指前一層到這一層完全連結。

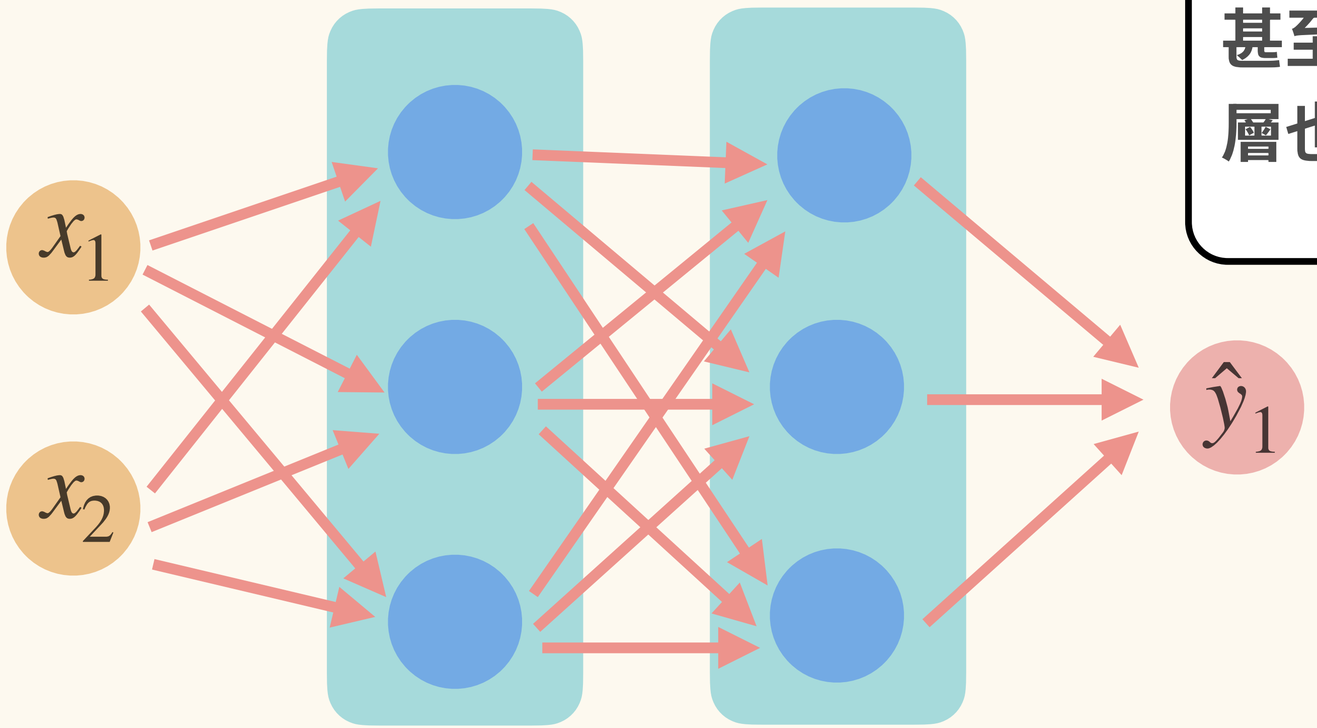
前一層輸出到下一層**每一個**神經元

開始打造神經網路!

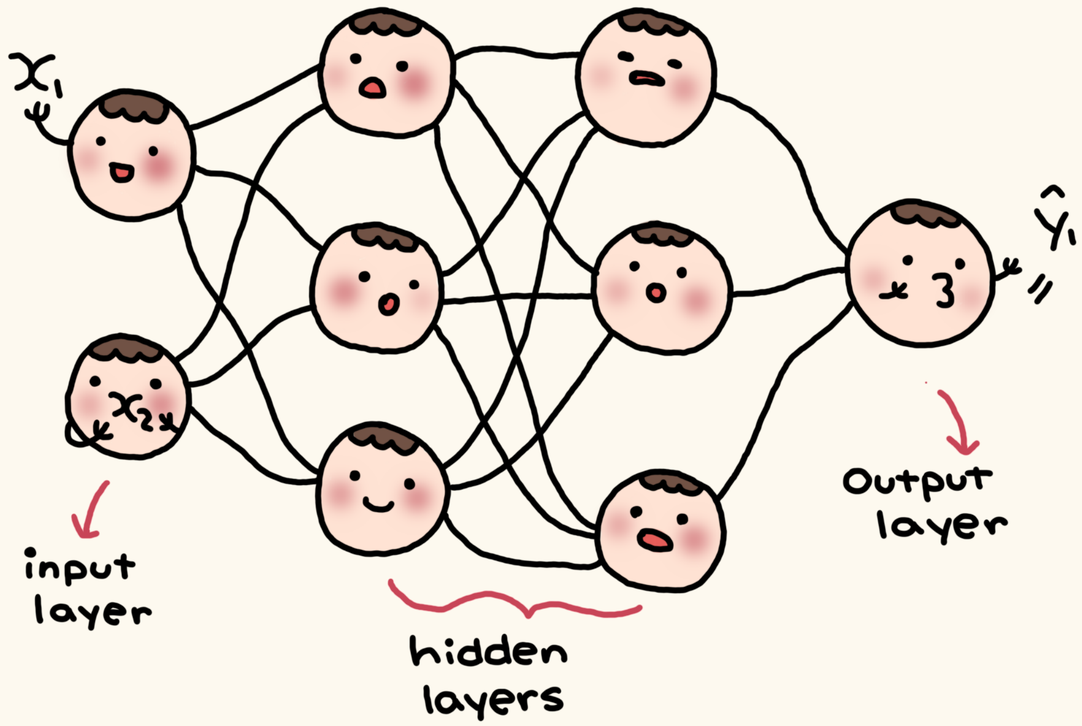


前一層輸出到下一層**每一個**神經元

開始打造神經網路!

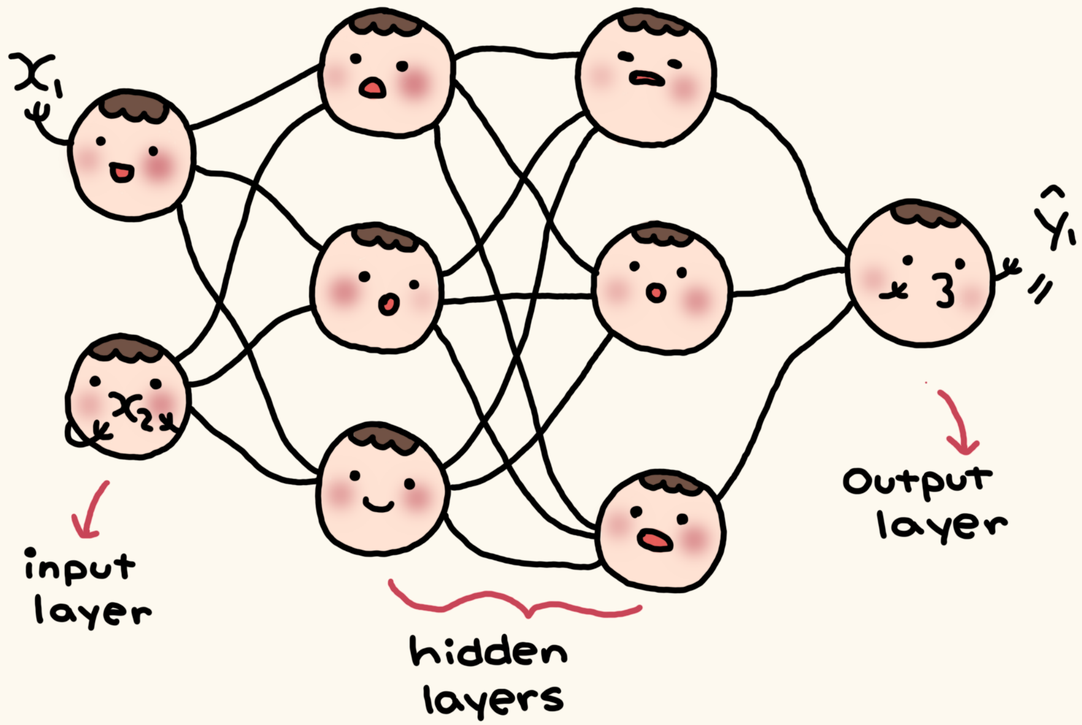


打造 DNN 就這麼簡單!



- 只需要決定兩組數字, 神經網路就決定了!
  1. 幾層隱藏層
  2. 每層幾個神經元
- 層層之間的神經元是完全連結的。

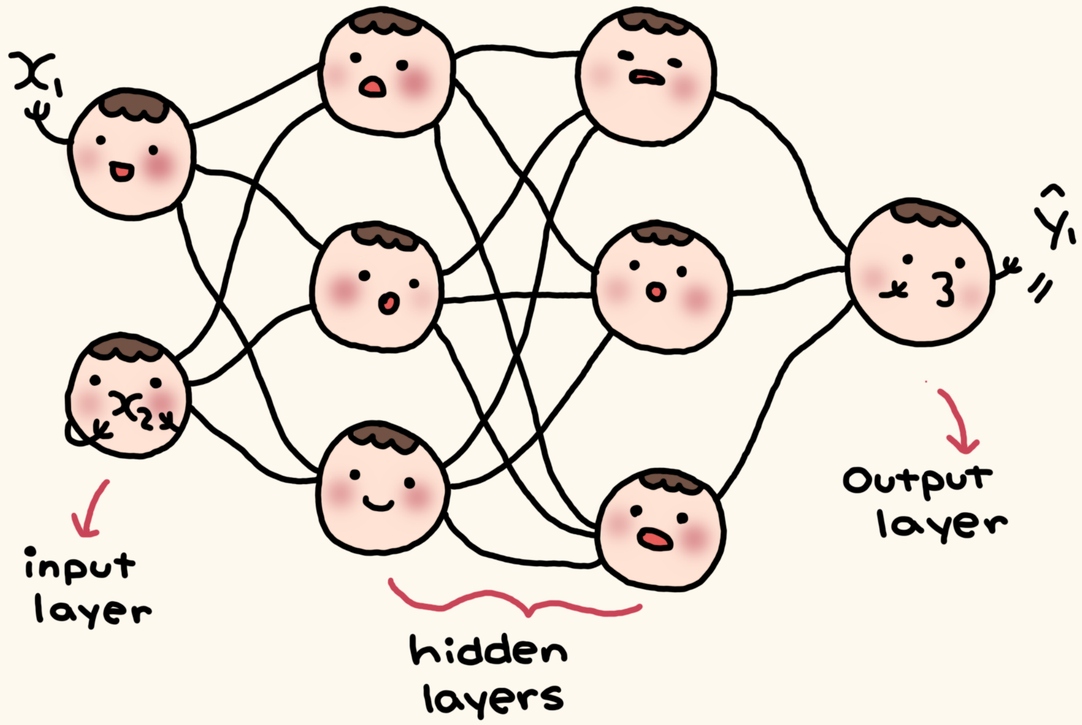
函數學習機建造完成!



我們會把所有要學習的參數, 包括所有的權重、所有的偏值, 合併起來叫做:

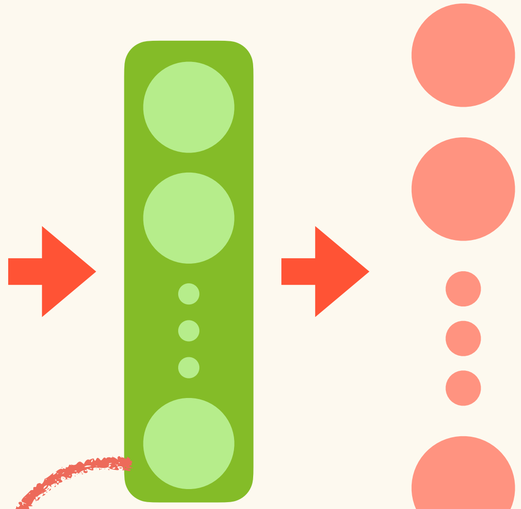
$$\theta = \{w_1, w_2, w_3, w_4, \cdots, b_1, b_2, \cdots\}$$

函數學習機建造完成!



當決定了權重、參數, 也就是一組 θ 的值, 我們輸入任意的數值, 神經網路就會吐出一個輸出給我們了!

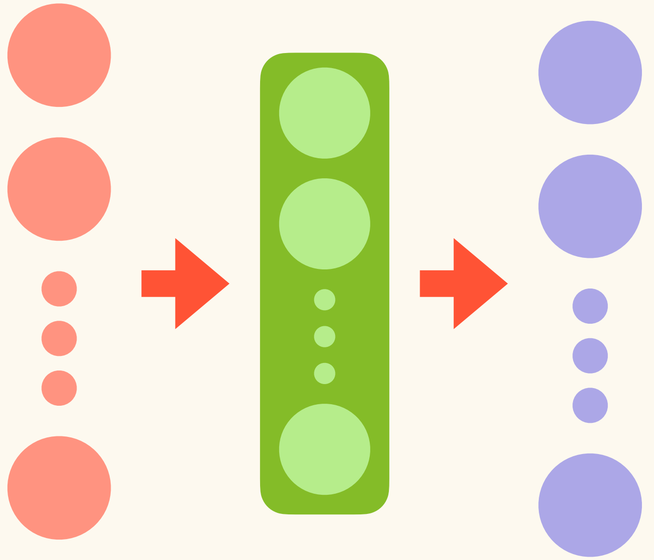
每一層神經網路把輸入轉為另一個 tensor



輸出通常是一個向量。

第 1 層隱藏層, 基本上就是放一堆神經元在裡面

每一層神經網路把輸入轉為另一個 tensor



每一層的輸出，都可以當成電腦對輸入的**理解**

神經網路暗黑學習法!

神經網路是個近乎全能的暗黑函數學習法!

神經網路暗黑學習法!

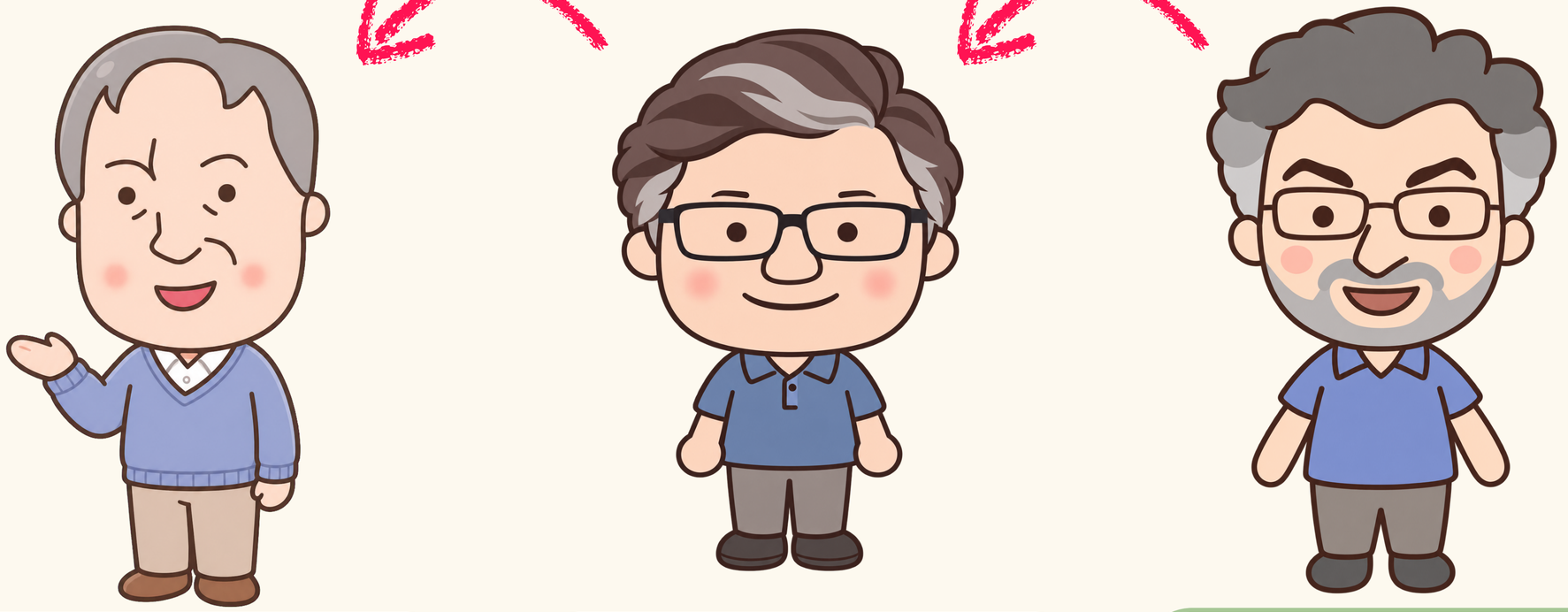
還有數學定理證明
「一個隱藏層」的
神經網路就能學會
你要學的函數!

Universal Approximation Theorem

深度學習三巨頭

博士後學生

博士後共事

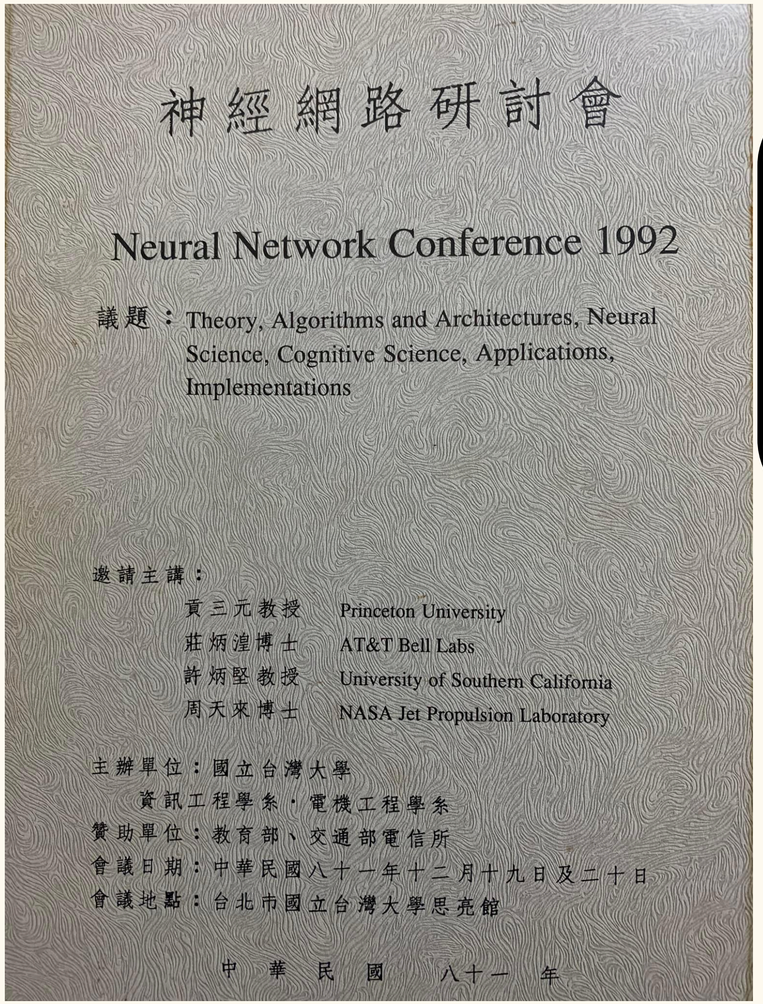


Geoffrey Hinton

Yann LeCun

Yoshua Bengio

在台灣也有不少人投入研究



神經網路研討會

Neural Network Conference 1992

議題：Theory, Algorithms and Architectures, Neural Science, Cognitive Science, Applications, Implementations

邀請主講：

貢三元教授	Princeton University
莊炳湟博士	AT&T Bell Labs
許炳堅教授	University of Southern California
周天來博士	NASA Jet Propulsion Laboratory

主辦單位：國立台灣大學
資訊工程學系、電機工程學系
贊助單位：教育部、交通部電信所
會議日期：中華民國八十一年十二月十九日及二十日
會議地點：台北市國立台灣大學思亮館

中 華 民 國 八十一 年

1992 在台大的神經網路研討會，當年對神經網路有興趣的大概都去了！

* 感謝 Jason Tsai 提供照片

魔法光茫不再!?

然後它就死掉了...

有一陣子如果研究計畫說要
研究「神經網路」, 那計畫還
沒送出就可以確定不會通過!

為什麼上個世紀神經網路沒有預期成功?

複雜的軟體

大量的數據

電腦計算能力

Yann LeCun

到了現在, 情況完全不一樣了!

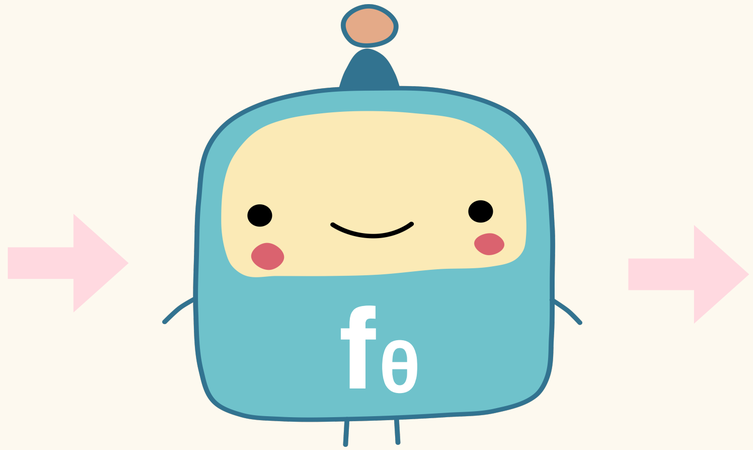
比起上個世紀, 要寫個
強強的深度學習程式,
真的太容易了!

03.

我們和真實的距離

神經網路架好之後，通常是先「初始化」參數值

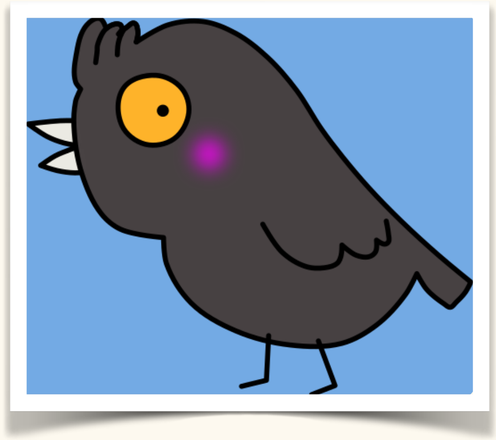
決定好這些參數的值，就會出現一個函數。

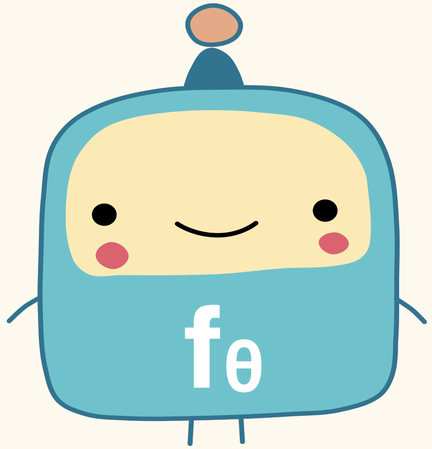


白話文就是，「這個神經網路可以動了」…

不過沒未經訓練的模型, 當然不準!

只是, 很有可能會發生這種情況...





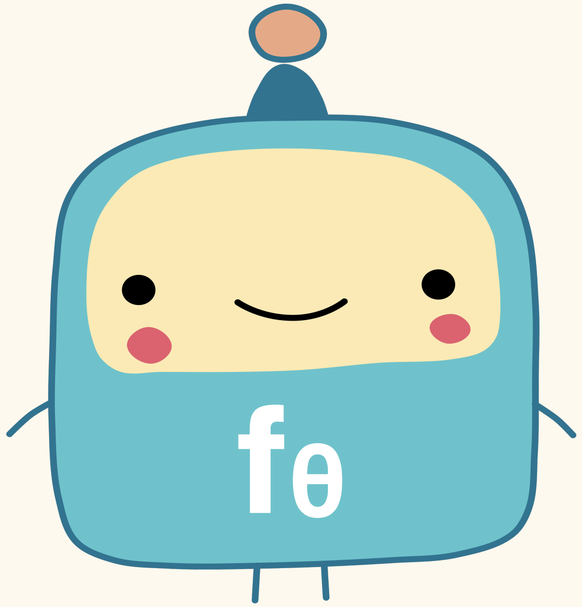
p_1 0.16

p_2 0.12

p_3 0.72 家八哥

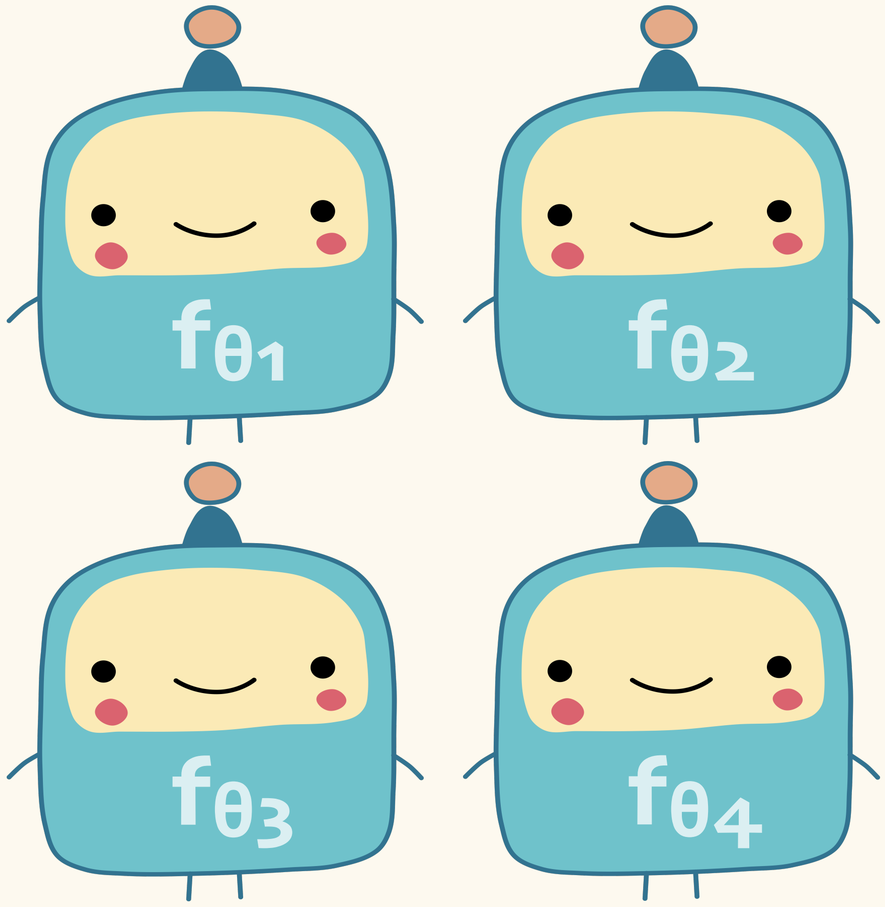
唬爛也要有個限度, 這該怎麼辦呢?

訓練 (學習)



注意我們的參數, 也就是 θ 決定了, 我們的函數學習機學出的函數就固定了 (也就是每張照片會回答什麼就決定了)。

函數空間



當我們用某個機器學習的方式, 打造了一台函數學習機, 這時需要訓練、調整的參數 θ 都決定了。

這時要只要 θ 參數值決定, 函數就決定了。注意就算固定了模型, 還是有無限多個可能的函數。

$$\{f_\theta\}$$

Cost Function/Loss Function

我們會用一個叫 loss function 的, 計算訓練資料中, 我們呆萌 AI 機器人的答案, 和正確答案有多大的差距。

自然我們希望 loss function 的值是越小越好。

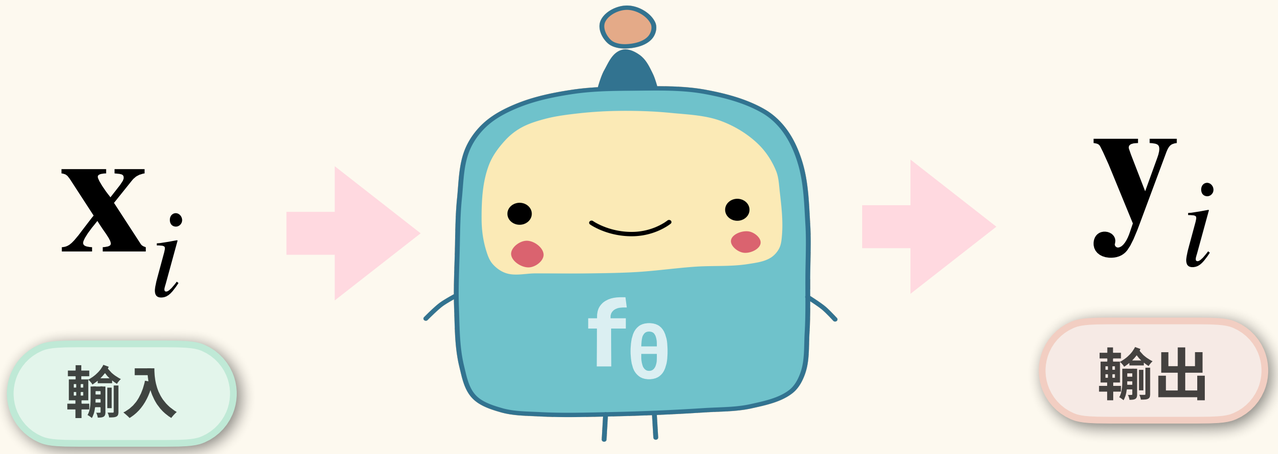
Loss Function

$$\{(x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)\}$$

意思是輸入 x_i, 正確答案應該是 y_i。

Loss Function

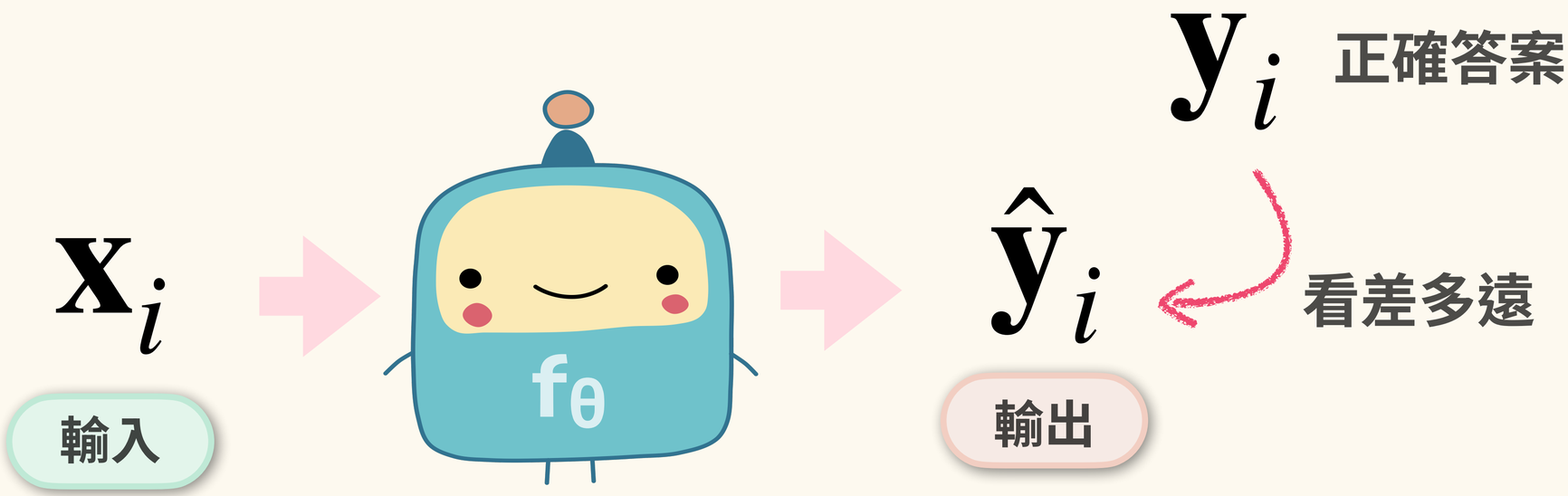
任何一組參數 θ, 就會給定一個函數 f_θ。這個函數 f_θ 對任意的輸入 $\mathbf{x}_i$ 都會給出一個值 $\hat{\mathbf{y}}_i$。



$$f_\theta(\mathbf{x_i}) = \hat{\mathbf{y}}_\mathbf{i}$$

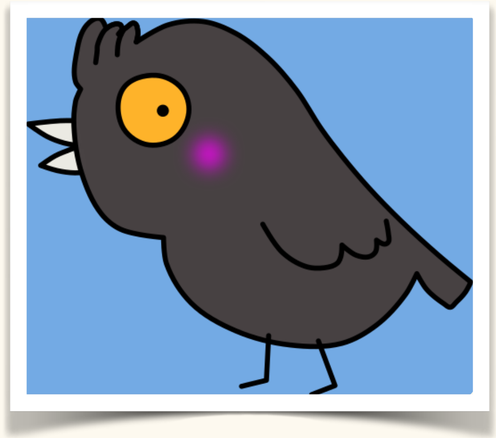
Loss Function

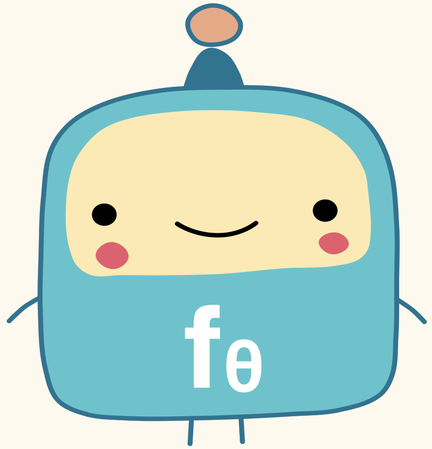
我們當然希望神經網路給出的 $\hat{y}_i$, 和正確答案 y_i 差距是越小越好!



$$f_\theta(x_i) = \hat{y}_i$$

比如說, 八哥辨識...





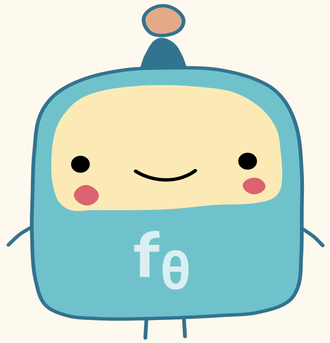
p_1 土八哥

p_2 白尾八哥

p_3 家八哥

比如說, 八哥辨識...

$\mathbf{x}_1$

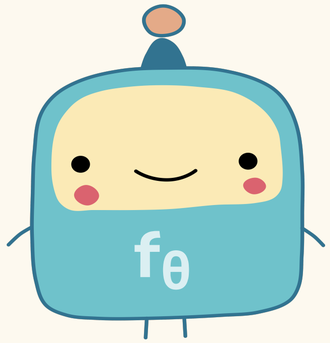


$$\begin{bmatrix} 0.23 \\ 0.43 \\ 0.34 \end{bmatrix}$$

$\hat{\mathbf{y}}_1$

$\mathbf{y}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$

$\mathbf{x}_2$

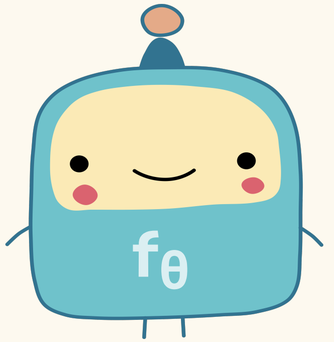


$$\begin{bmatrix} 0.36 \\ 0.28 \\ 0.37 \end{bmatrix}$$

$\hat{\mathbf{y}}_2$

$\mathbf{y}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$

$\mathbf{x}_3$



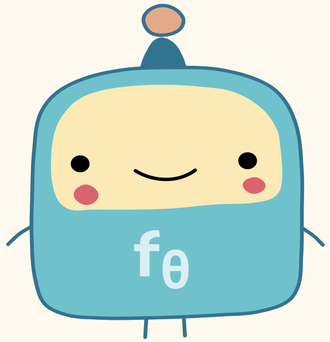
$$\begin{bmatrix} 0.46 \\ 0.28 \\ 0.27 \end{bmatrix}$$

$\hat{\mathbf{y}}_3$

$\mathbf{y}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

舉例來說，這差多遠呢?

$\mathbf{x}_1$



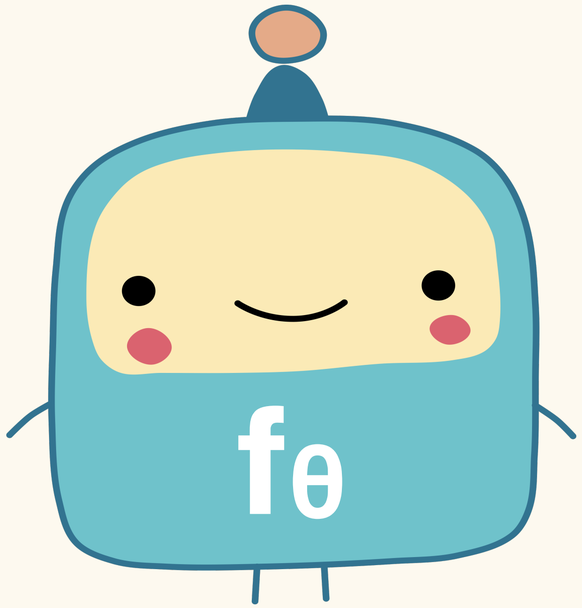
$$\begin{bmatrix} 0.23 \\ 0.43 \\ 0.34 \end{bmatrix}$$

$\hat{\mathbf{y}}_1$

$$\mathbf{y}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$(1 - 0.23)^2 + (0 - 0.43)^2 + (0 - 0.34)^2 = 0.89$$

評分學習效果的「損失函數」



我們會定義個 loss function,
看看我們的神經網路考考古題
的時候, 和正確答案差多少?

$$L(\theta)$$

Loss Function 就是估算答題好壞的

Loss function 就是計算神經網路給的答案, 和正確答案差距多少的函數。例如說以下是常見的 loss function:

$$L(\theta) = \frac{1}{2k} \sum_{i=1}^{k} \|\mathbf{y_i} - f_\theta(\mathbf{x_i})\|^2$$

Loss Function

其實就是計算和正確答案的差距!

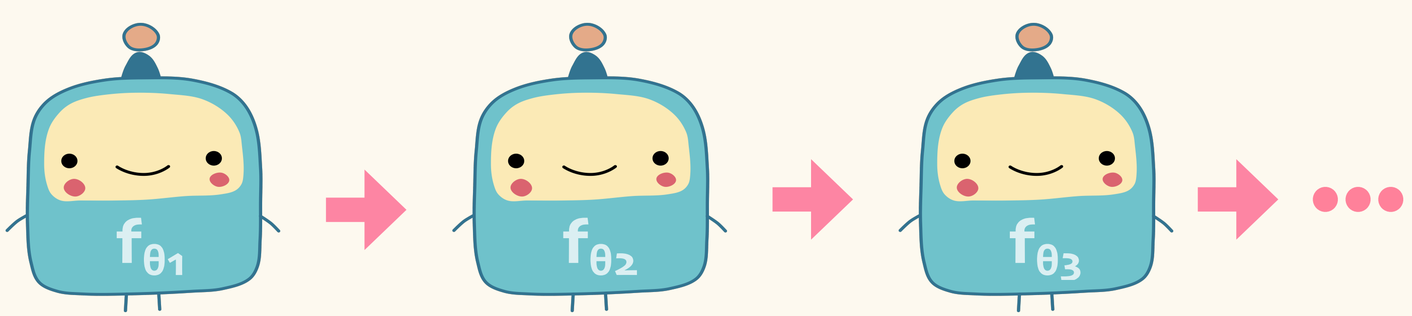
$$L(\theta) = \frac{1}{2k} \sum_{i=1}^{k} \| \mathbf{y_i} - f_\theta(\mathbf{x_i}) \|^2$$

函數學習機給的答案

正確答案

我們希望誤差越小越好!

所謂「訓練」就是想辦法讓 loss 越來越小...



$$L(\theta_1) > L(\theta_2) > L(\theta_3) > \cdots$$

最終找到「最好的」參數

目標是找到一組

$$\theta^*$$

這組參數代入 loss function

$$L(\theta^*)$$

值是最小的 (也就是**誤差最小**)。

用的方法叫梯度下降法

基本上就是用

gradient descent

梯度下降法

因神經網路的特性, 也叫

backpropagation

反向傳播法

04.
Gradient Descent:
萬用學習法

函數空間

$$\theta^*$$

我們的目標是要挑出一組最好的 θ^*，意思就是這樣做出來的 f_{θ^*} **和目標函數最接近**。

參數調整

那是怎麼調整的呢? 對於某個參數 w 來說, 其實就是用這「簡單的」公式:

$$-\eta \frac{\partial L}{\partial w}$$

參數調整

我們來破解函數學習機學習的秘密!

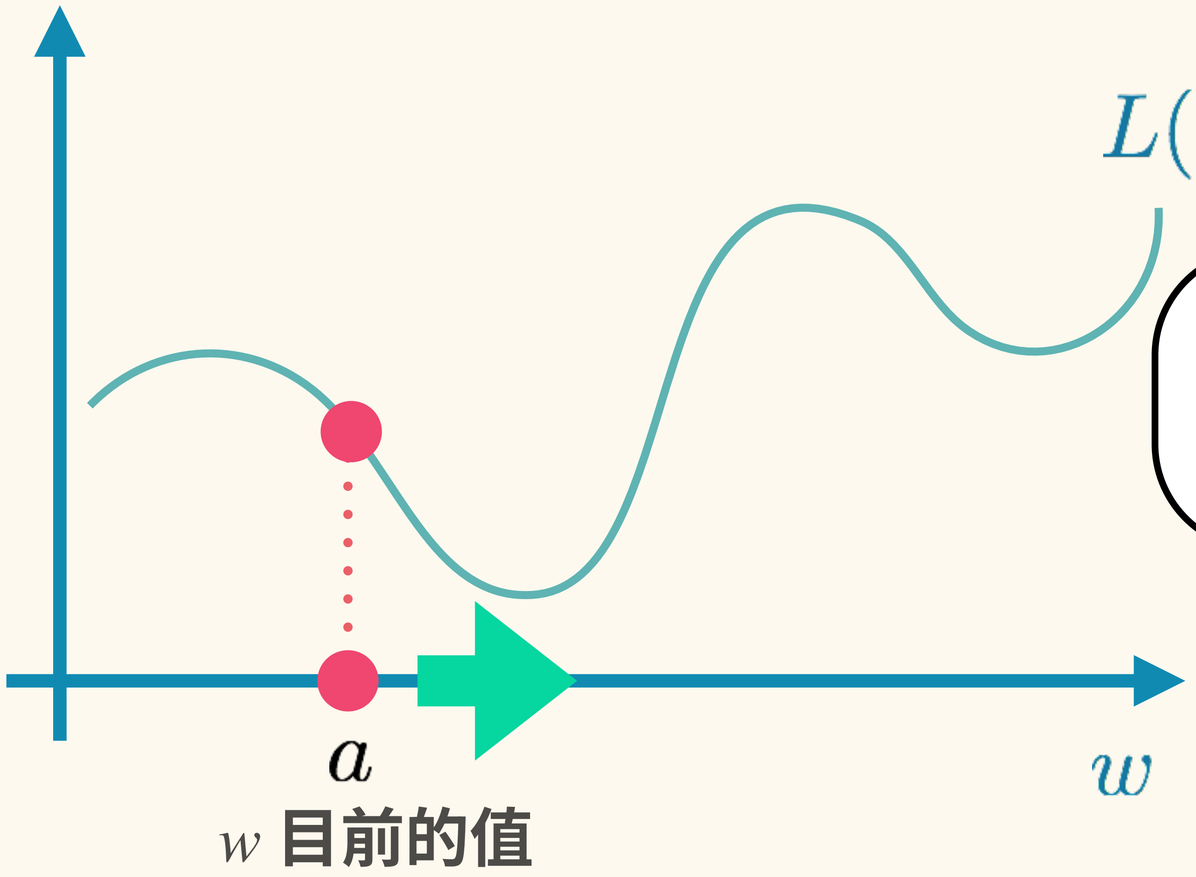
記得 loss function L 是所有權重和偏值 $w_1, w_2, b_1, \ldots$ 等等參數的函數。

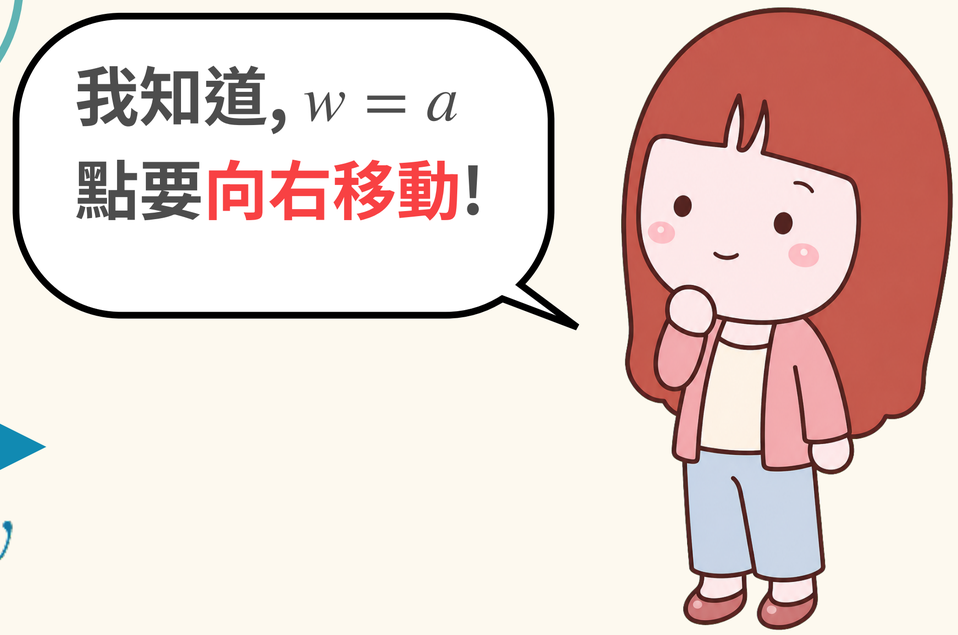
假裝只有一個參數!

數學家都會先簡化問題。

假裝只有一個參數!

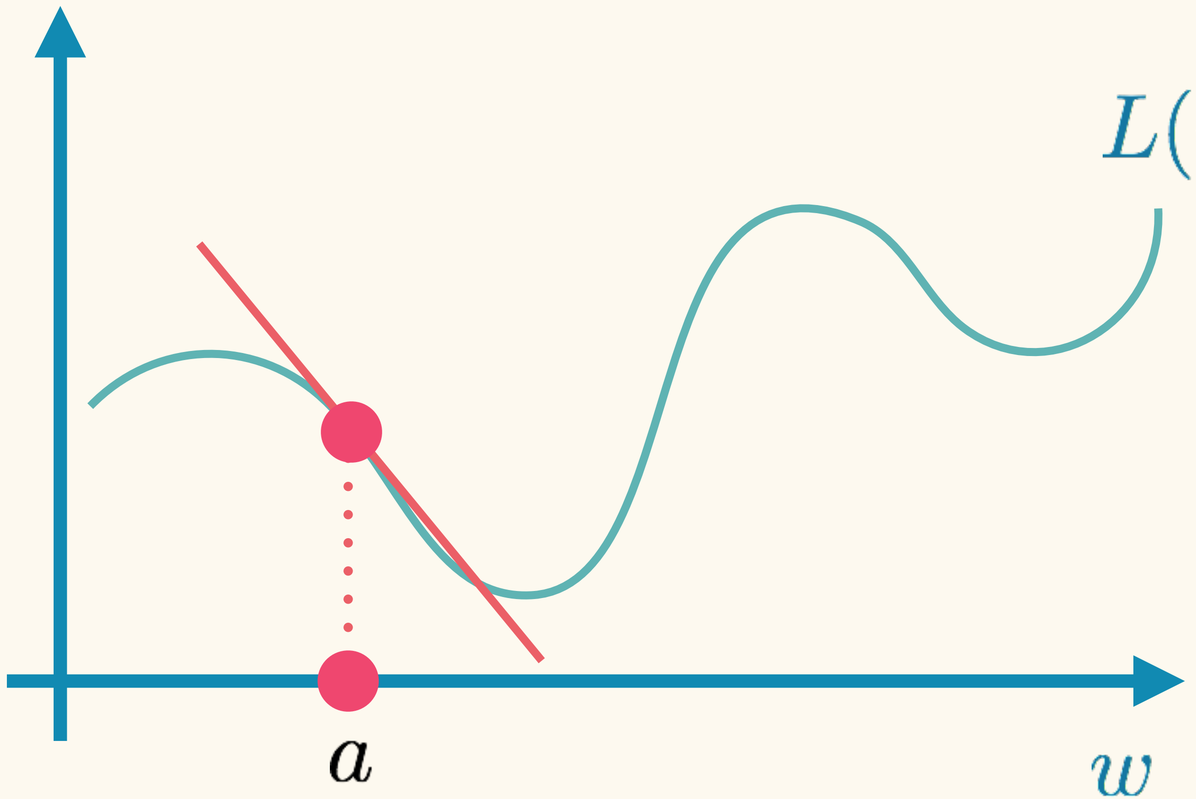
假裝只有一個參數!



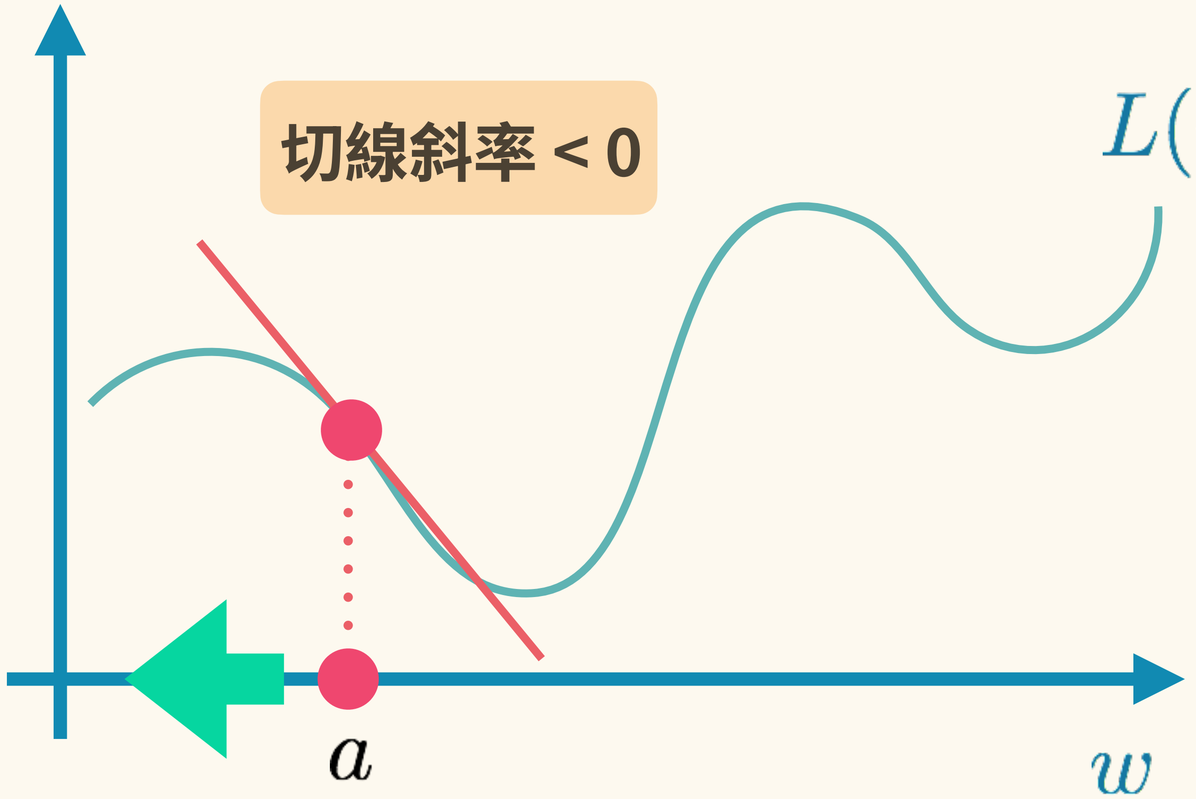


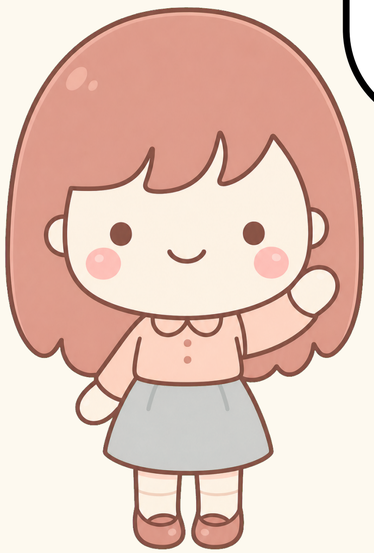
電腦怎麼知道要往哪走?

電腦怎麼知道要往哪走?



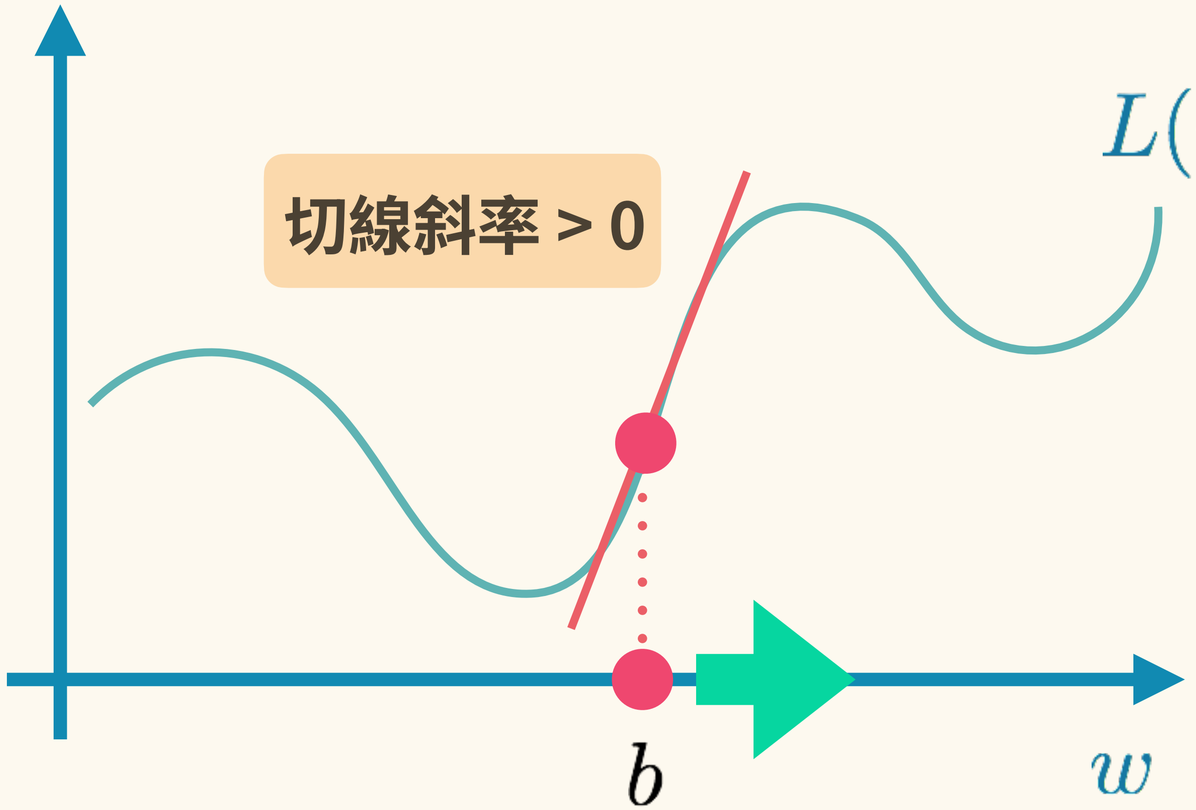
電腦怎麼知道要往哪走?

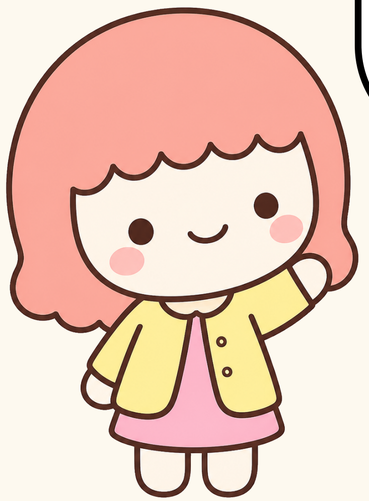




切線斜率是負的, 指的方向是負向。

電腦怎麼知道要往哪走?





切線斜率是正的, 指的方向是正向。

電腦怎麼知道要往哪走?

切線**斜率指的方向**和我們應該要走的**極小值方向是相反**的!

事實上切線斜率指的方向正是 (局部) 極大值的方向!

切線斜率炫炫的符號

在 $w = a$ 點的切線斜率符號是這樣:

$$L'(a)$$

對任意的 w 點來說, 我們寫成函數形式是這樣:

$$L'(w) = \frac{dL}{dw}$$

往 (局部) 極小移動!

在 $w = a$ 我們可以調整新的 w 值為:

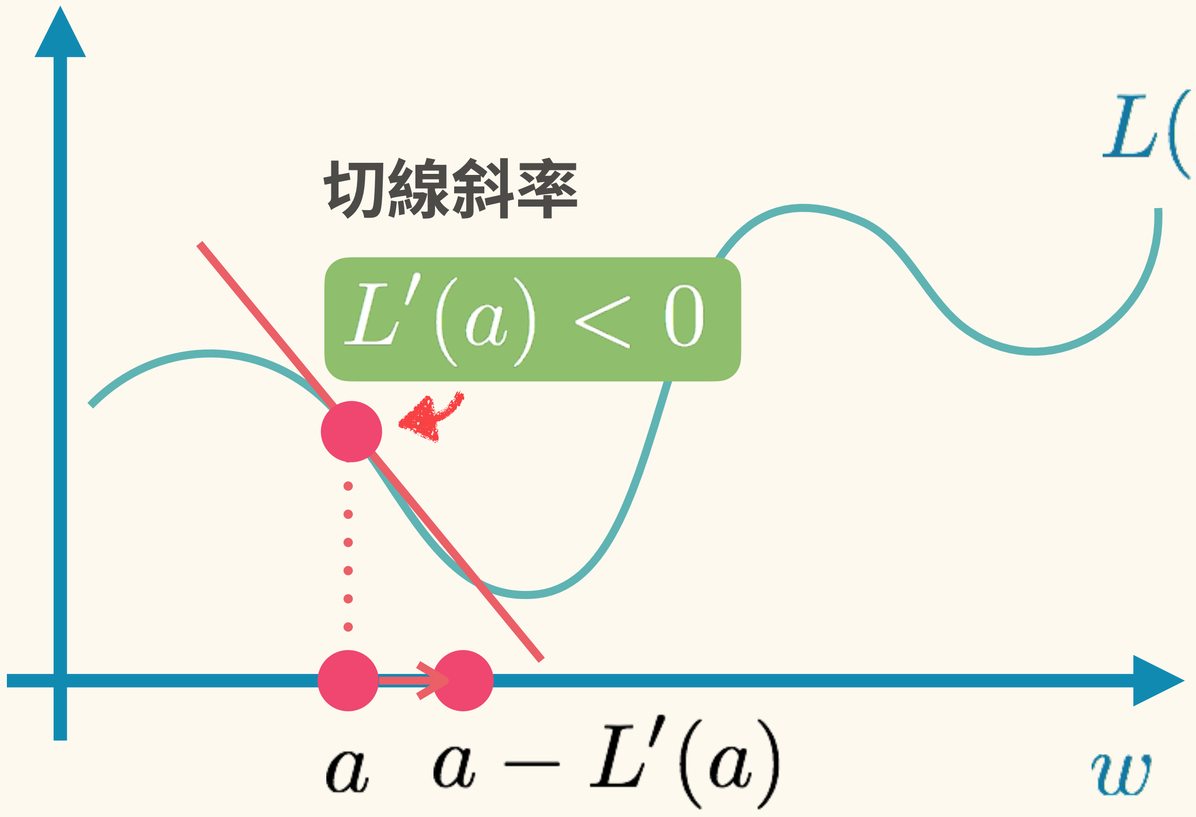
$$a - L'(a)$$

對任意的 w 點來說, 我們會有這樣子的公式去調整 w 的值。

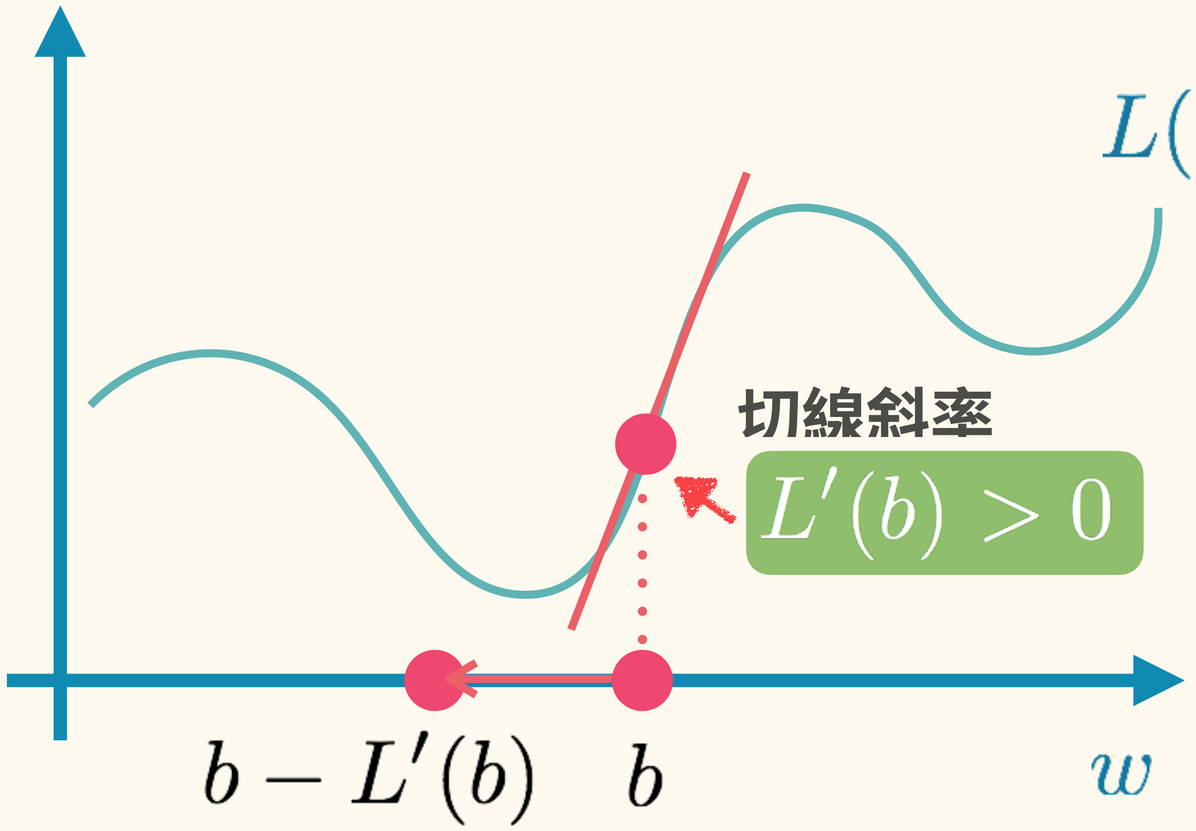
$$w - \frac{dL}{dw}$$

是不是真的可以呢? 有點緊張。

往 (局部) 極小移動!



往 (局部) 極小移動!

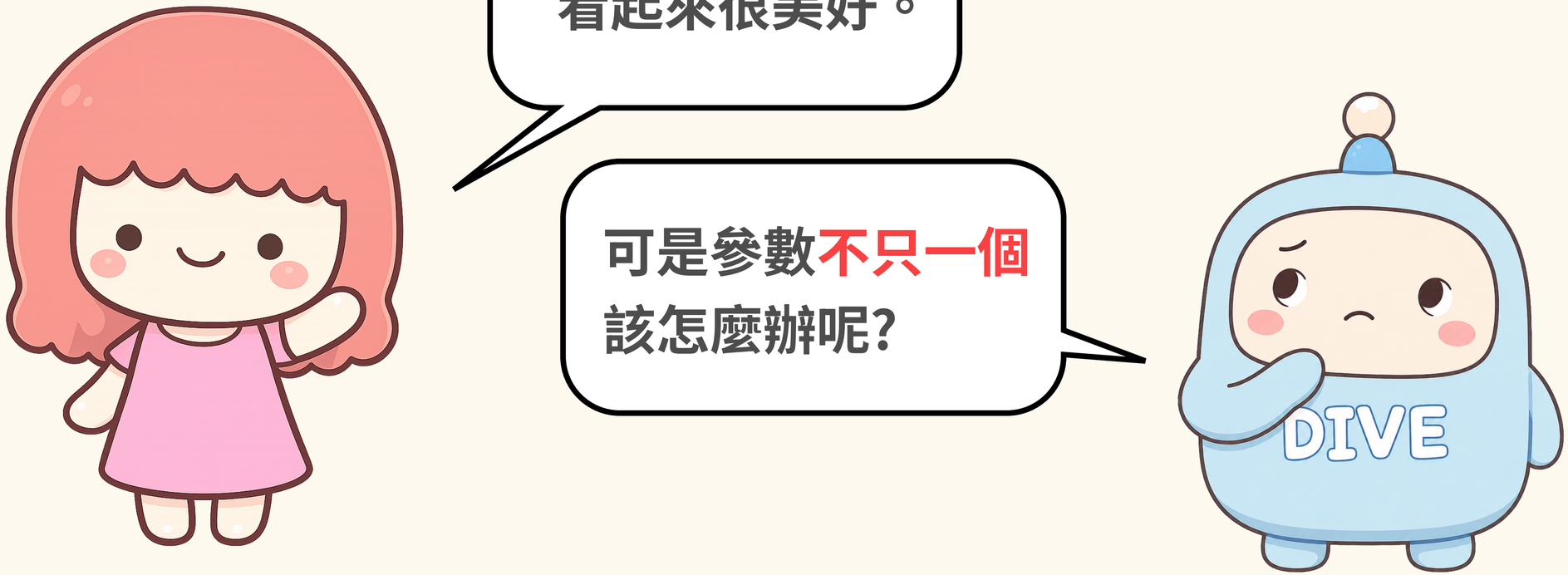


Learning Rate

$$w - \eta \frac{dL}{dw}$$

為了不要跑過頭, 我們不要一次調太大, 我們會乘上一個小小的數, 叫 Learning Rate。

不只一個參數怎麼辦呢？



數學家比我們想像中邪惡

假裝只有一個參數

假設我們的神經網路有三個參數 w_1, w_2, b_1, 而 loss function 長這樣:

$$L(w_1, w_2, b_1) = (b_1 + 2w_1 - w_2 - 3)^2$$

設我們把這個神經網路初始化, 各參數的值如下:

$$w_1 = 1, w_2 = -1, b_1 = 2$$

假裝只有一個參數

我們現在**假裝只有** w_1 **一個參數!**

不要問我為什麼...

假裝只有一個參數

$$L(w_1, w_2, b_1) = (b_1 + 2w_1 - w_2 - 3)^2$$

$$w_1 = 1, w_2 = -1, b_1 = 2$$

$$L_{w_1}(w_1) = L(w_1, -1, 2) = 4w_1^2$$

L 中只要保留 w_1 這個變數, 其他值都直接代入! 於是 L 當場只剩一個參數!

假裝只有一個參數

$$w_1 \leftarrow w_1 - \eta \frac{dL_{w_1}}{dw_1}$$

然後我們就可以用一個變數調整的方式, 去調整我們的權重!

偏微分

$$\frac{\partial L}{\partial w_1} = \frac{dL_{w_1}}{dw_1}$$

多變數函數, 假裝只有一個變數的微分方式就叫**偏微分**。

意思是我們會把 w_1 調整為

$$w_1 - \eta \frac{\partial L}{\partial w_1}$$

其實就是一一調整!

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$w_2 \leftarrow w_2 - \eta \frac{\partial L}{\partial w_2}$$

$$b_1 \leftarrow b_1 - \eta \frac{\partial L}{\partial b_1}$$

三個式子寫在一起

$$\begin{bmatrix} w_1 \\ w_2 \\ b_1 \end{bmatrix} \leftarrow \begin{bmatrix} w_1 \\ w_2 \\ b_1 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial b_1} \end{bmatrix}$$

Gradient 梯度

$$\begin{bmatrix} w_1 \\ w_2 \\ b_1 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial b_1} \end{bmatrix}$$

這關鍵的向量我們稱為 L 的**梯度 (gradient)**，記為:

$$\nabla L$$

Gradient Descent

$$\begin{bmatrix} w_1 \\ w_2 \\ b_1 \end{bmatrix} - \eta \nabla L$$

調整參數的公式就可以寫成這樣, 我們稱這個方法為:

Gradient Descent

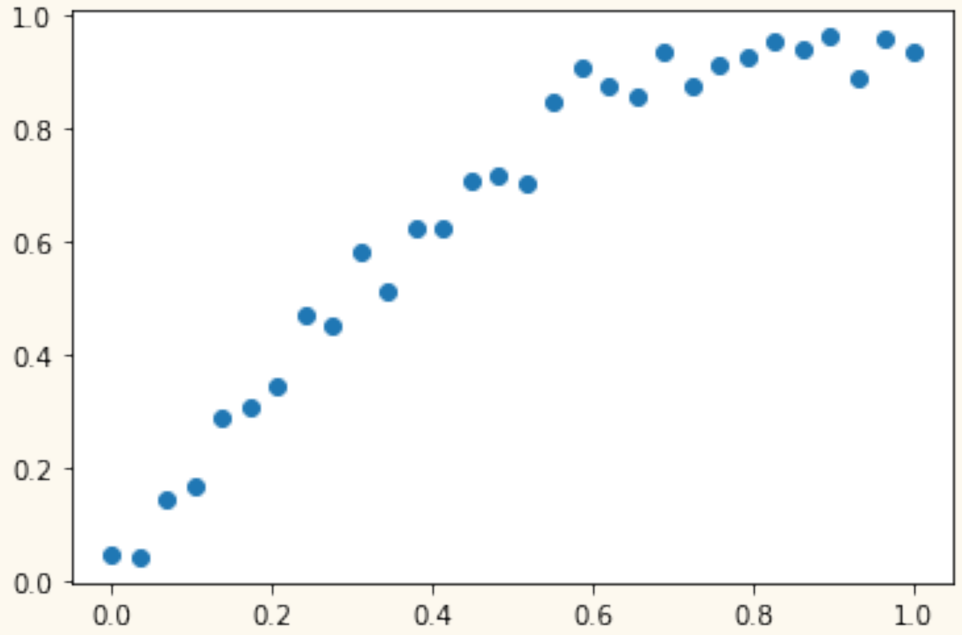
Gradient Descent

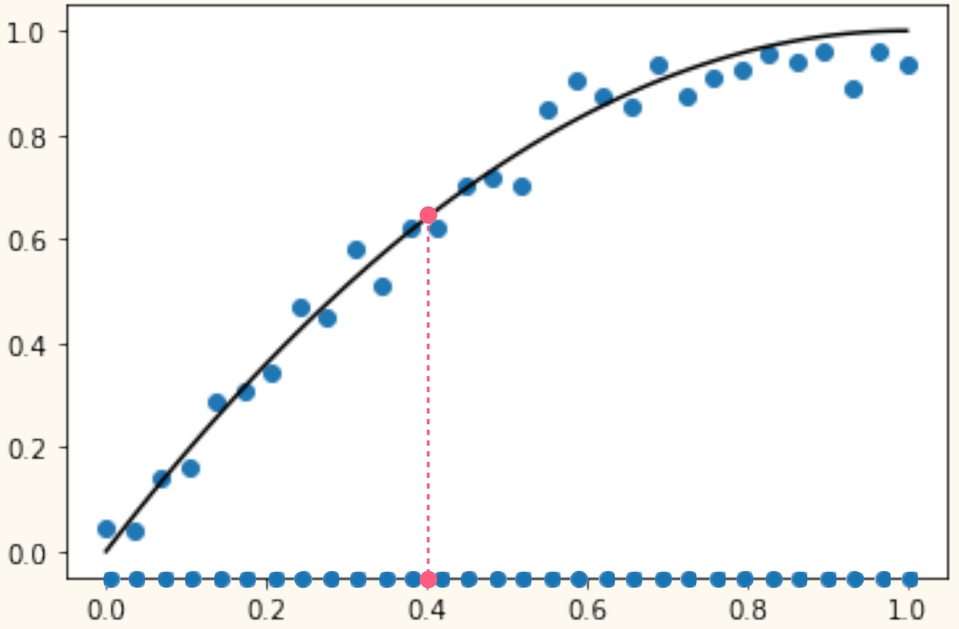
不管你的機器學習函數學習機是怎麼架的, 不管你選什麼 loss function, 你都可以使用 gradient descent 去訓練你的呆萌 AI 模型!

進階討論

一些有趣、深入的主題。

AI 基本上就是補足沒有學過的點



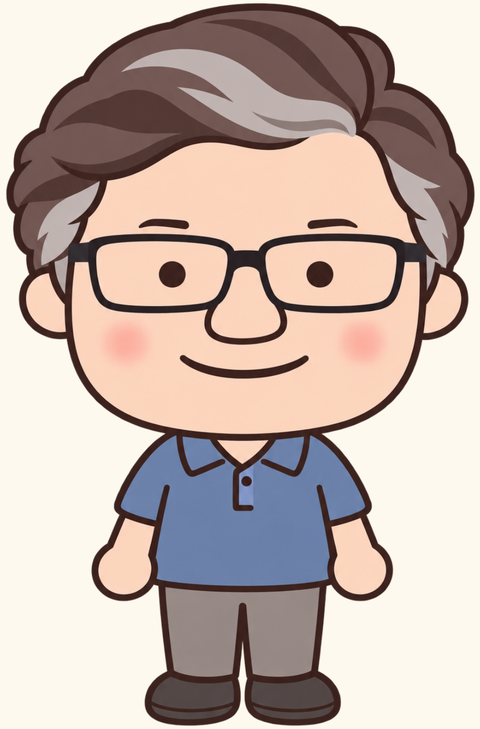


所以很多人就說這只是 curve fitting, 而且是內插法。

事實上高維度是外插!

Randall Balestriero, Jerome Pesenti, and Yann LeCun, “Learning in High Dimension Always Amounts to Extrapolation,” 2021.

https://arxiv.org/abs/2110.09485



05.

打造第一個神經網路

真的打造一個自己的神經網路!

用全連結神經網路做手寫辨識!

https://yenlung.me/AI01

TensorFlow

TensorFlow

現在我們準備一起打造一個神經網路, 我們準備用有名的 TensorFlow 這個 Python 套件。

MNIST 數據集

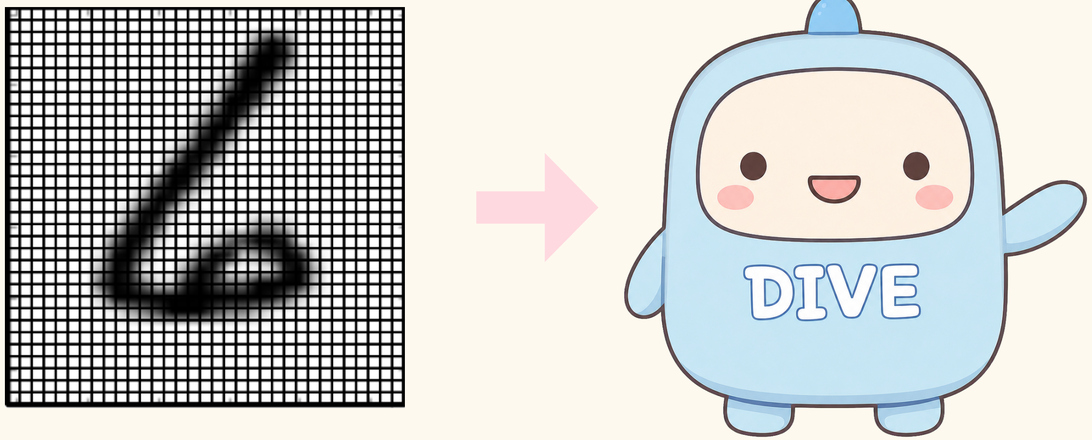
我們介紹一個非常有名的數據集。

Modified

MNIST

美國國家標準暨技術研究院

把我們的問題, 化為函數

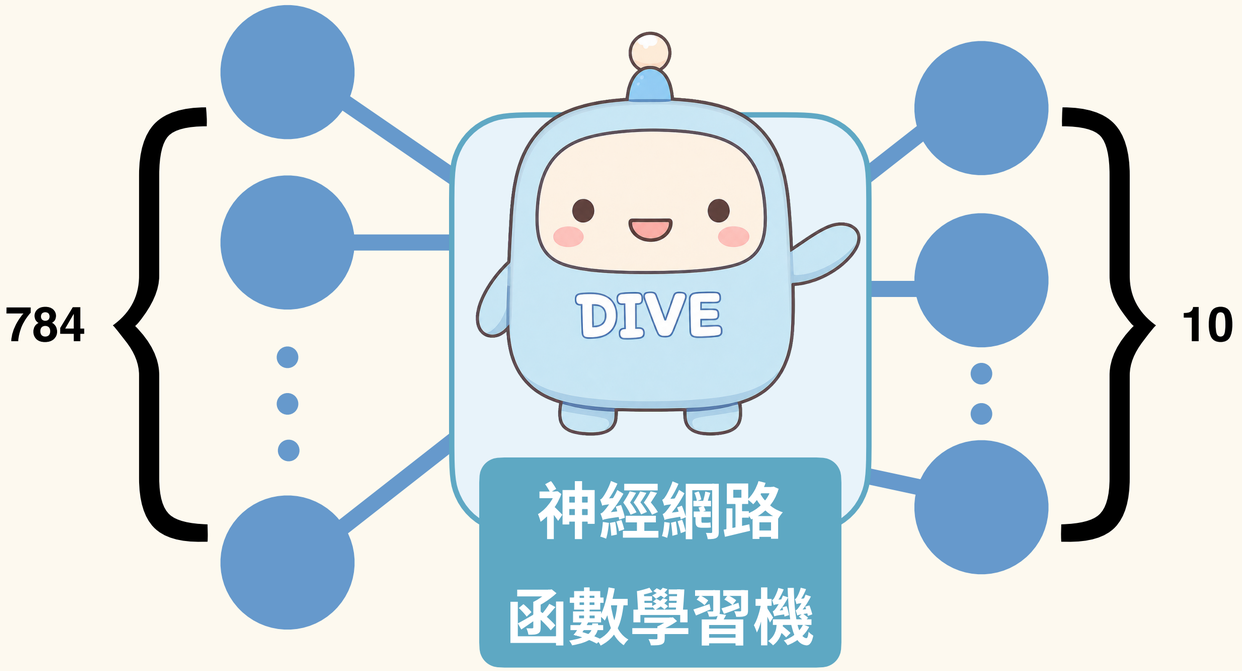


把我們的問題, 化為函數。比如說我們想做手寫辨識, 就是輸入一個掃描的手寫數字, **輸入**電腦, 希望電腦**輸出**這個數字是什麼。

One-hot encoding

我們在做分類問題時, 常會用 one-hot encoding。

確立函數的樣子



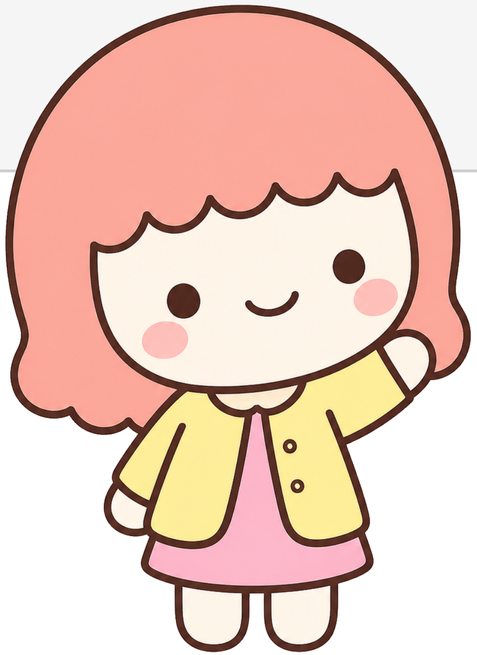
輸入是一張 28×28 的圖，「拉平」是 784 維的向量。

輸出 10 個類別，也就是 10 維的向量。

讀入數據分析基本套件

In [1]:

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```



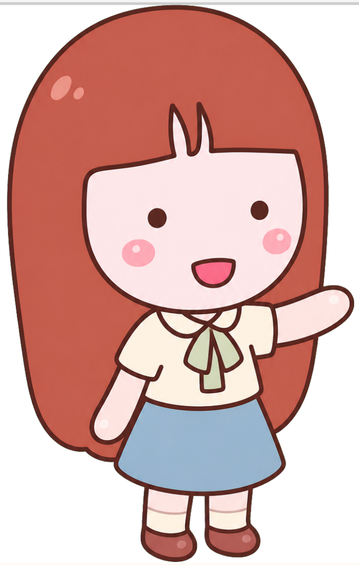
這裡大概是所有數據分析共通的。

打造神經網路的基本套件

做 one-hot encoding 的工具

In [2]:

```
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```



- `Sequential`: 標準打造一台深度學習函數學習機的方式。
- `Dense`: 做全連結的隱藏層。
- `SGD`: 標準的 gradient descent。

讀入 MNIST 數據集

In [3]:

```
from tensorflow.keras.datasets import mnist
```

In [4]:

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

數據處理

In [12]:

```
x_train = x_train.reshape(60000, 784)/255
x_test = x_test.reshape(10000, 784)/255
```

In [13]:

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

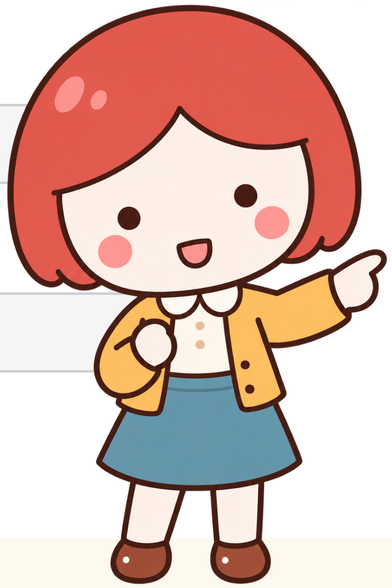
- **訓練資料**: 拉平、nomalization。
- **測試資料**: 做 one-hot encoding。

打造你的呆萌型 AI 機器人

```
In [14]:
model = Sequential()

In [15]:
model.add(Dense(100, input_dim=784, activation='relu'))

In [16]:
model.add(Dense(100, activation='relu'))

In [17]:
model.add(Dense(10, activation='softmax'))
```

假設我們做兩個隱藏層, 都是 100 個神經元、都用 ReLU 做 activation function。

輸出 10 維, 做 softmax。



就差不多只是告訴電腦, 我們怎麼設計神經網路的就結束了!

選擇 loss function、訓練方法

In [18]:

```
model.compile(loss='mse', optimizer=SGD(lr=0.087),
              metrics=['accuracy'])
```

- `loss`: 用哪個 loss function。
- `optimizer`: 我們用 SGD, 設 learning rate。
- `metrics`: 這裡是顯示目前正確率。

In [20]:

```
model.fit(x_train, y_train, batch_size=100, epochs=20)
```

- `batch_size`: mini batch 的大小。
- `epochs`: 訓練次數。

用測試資料試試

In [21]:

```
predict = model.predict_classes(x_test)
```

這樣 predict 就會是對所有測試資料的預測結果。

Q&A